

Acoustic Engine RPM Estimation via Digital Signal Processing

From Sampling Theory to a Real-Time Embedded Monitoring System

Adebanji Oluwatimileyin Adelowo

2026

Contents

1	Introduction and Motivation	4
1.1	The measurement problem	4
1.2	What this document covers	4
1.3	Scope, provenance, and reproducibility	5
2	Problem Definition	6
2.1	Formal statement	6
2.2	The chain of inference	6
2.3	The engine-state sub-problem	7
3	Terminology and Foundations: Signals, Sampling, and Aliasing	8
3.1	Continuous-time and discrete-time signals	8
3.2	The sampling theorem and aliasing	8
3.3	Why 44,100 Hz, and why not stay there	9
3.4	Choosing the analysis band and the decimated rate	9
4	Mathematical Foundations: the DFT and the FFT	11
4.1	From the DTFT to the DFT	11
4.2	Frequency resolution and the time–frequency trade-off	11
4.3	The FFT: Cooley–Tukey decimation in time	12
4.4	Spectral leakage and the need for a window	13
4.5	The STFT and time-varying spectra	15
5	Filtering and Decimation Theory	16
5.1	Why a filter must precede downsampling	16
5.2	The Butterworth response	16
5.3	The decimation actually performed	17
5.4	The anti-aliasing filter that SciPy actually uses	18
5.5	The decimation factor is out of specification	19
6	Approaches to Rotating-Machinery Frequency and Order Estimation	21
6.1	Order analysis: the organising idea	21
6.2	FFT peak-picking	21
6.3	Order tracking and the Vold–Kalman filter	22

6.4	Tacholless order tracking	22
6.5	Envelope and cepstral analysis	23
6.6	Where this system sits	23
7	Harmonic Analysis and the RPM Formula	25
7.1	Kinematics of the four-stroke cycle	25
7.2	Deriving the RPM formula	26
7.3	Why <code>EVENTS_PER_CYCLE = 4</code> , not 2	26
7.4	Verifying the order assignment against measured data	27
7.5	The margin, and why it is the system's central risk	28
8	Peak Detection and Frequency Tracking	29
8.1	The deployed peak finder	29
8.2	The duplicated raw-signal analysis	29
8.3	The RPM block and its guards	30
8.4	The adaptive <code>TunableFilter</code>	30
9	System and Software Architecture	32
9.1	Why concurrency is required	32
9.2	The six threads	32
9.3	The bounded queue and overflow policy	33
9.4	Locking discipline	33
9.5	The UART framed binary protocol	34
9.6	CRC-8 over $GF(2)$	35
9.6.1	The algebra	35
9.6.2	The bitwise loop as polynomial long division	36
9.6.3	Detection guarantees	36
9.7	The ACK handshake	37
9.8	Health monitoring and recovery	37
10	Code Implementation	38
10.1	Configuration constants	38
10.2	The <code>Datum</code> container	38
10.3	The pipeline	39
10.4	The recorder thread	39
10.5	The analyzer thread	40
10.6	Evolution across iterations	40
10.7	The extended library	41
10.7.1	Module structure	41
10.7.2	<code>Decimate.py</code>	42
10.7.3	<code>Fourier.py</code>	42
10.7.4	<code>EngineAnalysis.EngineState</code> : the TFLite classifier	43
10.7.5	<code>RTAudioStreamSimulator</code>	44
10.7.6	<code>Sensors/</code>	44
11	Experiments and Validation	45
11.1	Why a staged strategy	45
11.2	Level 1: DSP pipeline	45

11.3	Level 2: virtual ESP32	46
11.4	Level 3: on-device hardware	46
11.5	What the ladder does not cover	47
12	Results and Interpretation	48
12.1	Documented Level-1 results, verified	48
12.2	The order spectrum of <code>vol01.wav</code>	48
12.3	Chunk-level behaviour: the deployed configuration	50
12.4	Multi-order estimation: a measured alternative	50
12.5	Summary of findings	51
13	Limitations and Failure Cases	53
13.1	The fixed order assumption	53
13.2	Generalisation to other engines	53
13.3	Single dominant peak under multi-source interference	53
13.4	No temporal smoothing or state	54
13.5	Latency	54
13.6	Unintegrated components	55
13.7	Implementation-level defects	55
13.8	Validation gaps	55
14	Future Research Directions	57
14.1	Harmonic-template order estimation	57
14.2	Cepstral cross-check	57
14.3	Kalman and Vold–Kalman speed tracking	58
14.4	Tacholeless order tracking with ridge following	58
14.5	Integrating the TFLite classifier	58
14.6	Sensor fusion with the vibration channel	58
14.7	Efficiency and robustness work	59
14.8	Validation programme	59
15	References	60
16	Appendix A: Reproducing the Measurements	62

1 Introduction and Motivation

1.1 The measurement problem

Every reciprocating aircraft engine has a tachometer. On a certified aircraft that tachometer is a calibrated instrument, driven either mechanically from an accessory pad on the gearbox or electrically from a magneto-derived pulse train, and its indication is a primary flight instrument. It is also, from the point of view of anyone who wishes to *record* engine behaviour rather than merely display it, a closed system: the signal exists inside the aircraft's certified instrument loop and is not casually available to a data logger, a maintenance analyst, or a research student.

This creates a recurring practical gap. Light-sport and ultralight aircraft — the Tecnam P92 family with the Rotax 912 engine being a canonical example — are operated intensively in flight-training and club environments, accumulate engine hours quickly, and are maintained on condition rather than under a heavily instrumented programme. The information that would most usefully support that maintenance (how long the engine actually spent at each power setting, whether start-up transients are becoming rougher, whether the idle is drifting) is generated continuously by the engine and then discarded.

An acoustic sensor is an attractive way to close that gap, for three reasons.

It is non-invasive. A microphone in the cabin or the engine bay touches nothing. It requires no accessory drive, no tap into the ignition harness, no modification to a certified installation, and therefore no airworthiness approval for the sensor path itself. For an experimental or research installation this reduces the engineering effort from *months* to *an afternoon*.

It is cheap. A USB or I2S microphone and a Raspberry Pi Zero cost less than a single certified instrument connector. The economics matter: the value of engine condition data is realised only when it is collected from *many* airframes over *many* hours, and that is only affordable if the per-airframe cost is small.

The signal is genuinely there. A four-stroke reciprocating engine is a machine that converts a periodic sequence of combustion events into rotation. The pressure pulses in the exhaust, the mechanical impacts of valve seating, and the torsional reaction of the crankcase all repeat at frequencies rigidly locked to crankshaft angle. The airborne sound field is therefore not noise that happens to contain a signal; it is a *harmonic stack* whose fundamental spacing is a direct algebraic function of engine speed. Extracting engine speed from it is a well-posed inverse problem, not a heuristic.

1.2 What this document covers

This document is the technical and research record of a working system that performs this measurement. The system runs on a Raspberry Pi, records audio from a microphone, estimates engine RPM and a binary on/off engine state by digital signal processing, and transmits the result to an ESP32 microcontroller over a framed binary UART protocol. It was developed and exercised against the Tecnam P92 with the Rotax 912 — a four-cylinder, four-stroke, horizontally-opposed engine with a 2.43:1 reduction gearbox, rated 73.5 kW (100 hp) at 5800 rpm in its 912 ULS form [rotax912]. The P92 Echo airframe is offered with either the 80 hp Rotax 912 UL or the 100 hp 912 ULS [tecnam].

The document has two purposes that are held in tension deliberately.

The first is *pedagogical and derivational*. Sections 3 to 8 build the signal processing from first principles: the sampling theorem and why 44,100 Hz capture is decimated to 500 Hz; the discrete Fourier transform, its frequency resolution, and the Cooley–Tukey factorisation that makes it computable; spectral leakage and the Hann window; Butterworth and Chebyshev filter design and the aliasing condition that decimation must satisfy; and finally the harmonic arithmetic of a four-cylinder four-stroke engine that turns a measured frequency into a revolutions-per-minute figure. Each derivation terminates in a specific constant or line of code in the deployed system.

The second purpose is *critical and empirical*. Sections 11 to 13 report what happens when the system is actually run against recorded engine audio. The results are not uniformly favourable, and the document does not present them as such. The central design decision — a hard-coded harmonic index `EVENTS_PER_CYCLE = 4` — is shown, with measured spectra, to be a correct calibration for one recording and an exact factor-of-two error for another, and to fail on roughly one five-second chunk in five even within the recording it was tuned on. Establishing *why*, with quantitative evidence, is the most useful contribution this document can make, because it identifies precisely which piece of the architecture must change and what it must be replaced with.

1.3 Scope, provenance, and reproducibility

All code discussed is in the repository `aircraft-engine-monitor`. The deployed application is `rpi_test_final.py`. Three earlier iterations (`rpi_test.py`, `rpi_test_fft.py`, `rpi_test_new.py`) are retained and are discussed where the differences between them are informative about the design’s evolution. A larger modular research library, `audio_src/`, contains components (an adaptive tunable bandpass filter, a TensorFlow Lite engine-state classifier, a real-time audio stream simulator) that were developed alongside the deployed script but are not yet integrated into it.

Every numerical result reported in Sections 11 and 12 was produced by executing the repository’s own pipeline classes on the repository’s own simulation audio, on a machine running NumPy 2.0.0 and SciPy 1.13.1 [numpy; scipy]. Where this document reports a spectrum, an order amplitude, or a per-chunk RPM sequence, that number was measured, not assumed.

2 Problem Definition

2.1 Formal statement

Let $p(t)$ denote the acoustic pressure at the microphone diaphragm, and let $\Omega(t)$ denote the instantaneous crankshaft angular speed of the engine, expressed in revolutions per minute. The engine's rotation modulates the pressure field through combustion, gas exchange, and structural radiation, so that $p(t)$ carries information about $\Omega(t)$ but is also contaminated by propeller noise, airframe boundary-layer noise, wind, avionics, cabin conversation, and the microphone's own self-noise.

The estimation problem is:

Given a finite, uniformly sampled, quantised observation $x[n] = \mathcal{Q}\{p(nT_s)\}$, $n = 0, \dots, N-1$, with sampling interval $T_s = 1/f_s$, produce an estimate $\hat{\Omega}$ of the crankshaft speed averaged over the observation window, together with a binary state $\hat{s} \in \{0, 1\}$ indicating whether the engine is running.

Three constraints turn this from a textbook exercise into an engineering problem.

Real-time operation. The system must produce an estimate for every observation window while continuing to acquire the next one. If analysis of a window takes longer than the window itself, the system falls progressively behind and eventually loses data. This is a *throughput* constraint, and it is the reason the architecture is concurrent rather than sequential (Section 9).

Embedded hardware. The target is a Raspberry Pi Zero, Pi 3, or Pi 4. There is no GPU, floating-point throughput is modest, and the same processor must also service the audio device, the filesystem, and a serial port. This is a *computational cost* constraint, and it is the reason the signal is decimated by a factor of 88 before spectral analysis (Section 5).

A physical, not statistical, target. Ω is a continuous physical quantity with units. An estimator that is merely *correlated* with engine speed is not acceptable; the output must be an unbiased estimate in revolutions per minute, because a downstream consumer (a display, a logbook, a maintenance trend) will interpret it dimensionally. This is what distinguishes the problem from *acoustic classification*, where a learned decision boundary suffices.

2.2 The chain of inference

The system does not estimate Ω directly. It estimates it through a chain of four inferences, each of which is a potential failure point:

$$x[n] \xrightarrow{\text{(i) decimate}} x_d[m] \xrightarrow{\text{(ii) DFT}} |X_d[k]| \xrightarrow{\text{(iii) argmax}} \hat{f}_{\text{peak}} \xrightarrow{\text{(iv) order model}} \hat{\Omega}$$

Step (i) must not destroy or alias the frequencies of interest. Step (ii) must resolve them, which sets a lower bound on window length. Step (iii) must select the *correct* spectral component among several candidates. Step (iv) must know the *harmonic order* of the component that step (iii) selected.

Steps (i) and (ii) are solved problems and are treated here as classical theory correctly applied. Steps (iii) and (iv) are coupled: step (iv) assumes a fixed answer to a question that step (iii) does not actually answer. This coupling is the system's central weakness, and Sections 7, 12, and 13 examine it in detail.

2.3 The engine-state sub-problem

Alongside the continuous estimate, the system emits a binary engine-on flag. In `rpi_test_final.py` this is a threshold on the RPM estimate itself:

```
engine_status = 1 if rpm > 100 else 0
```

This is deliberately crude, and its crudeness is defensible: an engine that is not running produces no locked harmonic stack, so the peak-picker returns either zero (if the band is empty) or an arbitrary noise-driven frequency, and a 100 RPM threshold on the derived quantity rejects both. The research library contains a substantially more sophisticated alternative — a TensorFlow Lite classifier operating on the RFFT magnitude spectrum with a 70-window rolling majority vote — which is analysed in Section 10.

3 Terminology and Foundations: Signals, Sampling, and Aliasing

3.1 Continuous-time and discrete-time signals

An acoustic signal at the microphone is a continuous-time function $p : \mathbb{R} \rightarrow \mathbb{R}$. Digital processing requires a *discrete-time sequence*, an ordered set of numbers $x[n]$ indexed by $n \in \mathbb{Z}$. The transition between the two is the act of sampling, and the entire validity of everything downstream depends on it being performed correctly. The standard reference for the material in this and the following two sections is Oppenheim and Schaffer [oppenheim2010].

Ideal uniform sampling of $p(t)$ at interval T_s produces

$$x[n] = p(nT_s), \quad n \in \mathbb{Z},$$

with sampling frequency $f_s = 1/T_s$. A useful way to view this is through the *impulse-train modulation* model. Define

$$s(t) = \sum_{n=-\infty}^{\infty} \delta(t - nT_s),$$

so that $p_s(t) = p(t)s(t)$ is a train of weighted impulses carrying exactly the sample values. The Fourier transform of an impulse train of period T_s is itself an impulse train of period f_s in frequency:

$$S(f) = \frac{1}{T_s} \sum_{k=-\infty}^{\infty} \delta(f - kf_s).$$

Multiplication in time is convolution in frequency, so

$$P_s(f) = P(f) * S(f) = f_s \sum_{k=-\infty}^{\infty} P(f - kf_s).$$

This is the central result. **Sampling replicates the spectrum of the underlying continuous signal at every integer multiple of the sampling frequency.**

3.2 The sampling theorem and aliasing

Suppose $p(t)$ is *bandlimited*, meaning $P(f) = 0$ for $|f| \geq B$. The replicas in $P_s(f)$ are then non-overlapping if and only if

$$f_s - B > B \quad \Longleftrightarrow \quad \boxed{f_s > 2B}$$

This is the Nyquist criterion. Nyquist established the underlying rate limit in the context of telegraph signalling in 1928 [nyquist1928]; Shannon gave the reconstruction theorem its modern statement and proof in 1949 [shannon1949]. The frequency $f_s/2$ is the *Nyquist frequency*, and $2B$ is the *Nyquist rate*.

When the criterion holds, the baseband replica ($k = 0$) is a faithful, isolated copy of $P(f)$, and $p(t)$ can be recovered exactly by ideal lowpass filtering, giving the Whittaker–Shannon interpolation formula

$$p(t) = \sum_{n=-\infty}^{\infty} x[n] \operatorname{sinc}\left(\frac{t - nT_s}{T_s}\right), \quad \operatorname{sinc}(u) = \frac{\sin(\pi u)}{\pi u}.$$

When the criterion is violated, adjacent replicas overlap. A component at true frequency $f_0 > f_s/2$ is then indistinguishable, in the sampled data, from a component at the *folded* frequency

$$f_{\text{alias}} = \left| f_0 - f_s \cdot \operatorname{round}\left(\frac{f_0}{f_s}\right) \right|,$$

which lies in $[0, f_s/2]$. This is *aliasing*, and it is irreversible: once two distinct continuous-time frequencies map to the same discrete-time frequency, no amount of subsequent processing can separate them.

The practical consequence for this system is direct and severe. If a 340 Hz component from the propeller or the airframe were to alias down to 161 Hz, the peak-picker would see a spectral line in the middle of its engine search band that has nothing whatsoever to do with the engine, and would report a confident, completely wrong RPM. Preventing this is not an optimisation; it is a correctness requirement.

3.3 Why 44,100 Hz, and why not stay there

The acquisition rate is fixed by `SAMPLE_RATE = 44100` in `rpi_test_final.py`. This is not chosen for signal-processing reasons at all. It is chosen because it is the canonical consumer audio rate, universally supported by USB and I2S microphone hardware and by the ALSA and PortAudio stacks that `sounddevice` sits on. A capture rate that the hardware refuses to negotiate is worth nothing, and 44,100 Hz always negotiates.

Its Nyquist frequency of 22,050 Hz vastly exceeds anything the engine produces in the band of interest, so the acquisition itself is comfortably oversampled. But that oversampling is pure cost for the analysis stage:

- A five-second window contains $N = 5 \times 44100 = 220,500$ samples.
- An FFT on that window costs $\mathcal{O}(N \log_2 N) \approx 220,500 \times 17.75 \approx 3.9 \times 10^6$ arithmetic operations (equivalently $(N/2) \log_2 N \approx 2.0 \times 10^6$ butterflies).
- Of the resulting 110,250 non-negative-frequency bins, the analysis band $[10, 200]$ Hz occupies only $190/22050 \approx 0.86\%$ of the spectrum.

More than 99% of the transform is computed and discarded. On a Raspberry Pi Zero this is not free. The remedy is *decimation*: lowpass-filter to a bandwidth that still contains the whole engine band, then keep only every M -th sample.

3.4 Choosing the analysis band and the decimated rate

The band of interest is set in the peak finder as $[10, 200]$ Hz. This is derived from the engine’s own harmonic arithmetic, developed fully in Section 7, but the reasoning can be stated now. A

four-cylinder four-stroke engine produces two combustion events per crankshaft revolution, so its firing frequency is

$$f_{\text{fire}} = \frac{2\Omega}{60} = \frac{\Omega}{30} \text{ Hz.}$$

The Rotax 912 operates from a low idle of order 1400 rpm to a take-off limit of 5800 rpm [rotax912], and starting or shutdown transients traverse speeds well below idle. Over a working range of roughly 500 to 6000 rpm, the firing frequency spans 17 Hz to 200 Hz, and the *second* harmonic of firing — which Section 7 shows to be the component this system actually tracks — spans 33 Hz to 400 Hz. The [10, 200] Hz window therefore captures the firing fundamental across the whole speed range and its second harmonic across the low and middle range, while excluding the DC and infrasonic region below 10 Hz where microphone rumble, wind-buffet, and DC offset dominate.

Given a 200 Hz upper band edge, the Nyquist criterion demands only $f_s^{(d)} > 400$ Hz. The chosen value is `DECIMATED_RATE = 500`, providing a Nyquist frequency of 250 Hz and thus a 50 Hz guard band above the analysis edge. That guard band is what gives the anti-aliasing filter room to roll off, and Section 5 quantifies exactly how much attenuation it achieves.

The resulting economy is substantial. The five-second window shrinks from 220,500 samples to approximately 2,506, the FFT cost falls by a factor of roughly

$$\frac{220500 \times \log_2 220500}{2506 \times \log_2 2506} \approx \frac{3.91 \times 10^6}{2.83 \times 10^4} \approx 138,$$

and the fraction of computed bins that are actually used rises from 0.86% to $190/250 = 76\%$.

4 Mathematical Foundations: the DFT and the FFT

4.1 From the DTFT to the DFT

For a discrete-time sequence $x[n]$, the discrete-time Fourier transform is

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]e^{-j\omega n},$$

a continuous, 2π -periodic function of normalised angular frequency ω . It is not computable: it requires infinitely many samples and produces a continuum of values.

The *discrete Fourier transform* is its computable counterpart. For a finite sequence of length N ,

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j2\pi kn/N}, \quad k = 0, 1, \dots, N-1$$

with inverse

$$x[n] = \frac{1}{N} \sum_{k=0}^{N-1} X[k] e^{+j2\pi kn/N}.$$

Writing $W_N = e^{-j2\pi/N}$ for the primitive N -th root of unity, the DFT is the matrix–vector product $\mathbf{X} = \mathbf{W}\mathbf{x}$ with $\mathbf{W}_{kn} = W_N^{kn}$. The DFT is exactly the DTFT of the finite sequence, sampled at N equally spaced points $\omega_k = 2\pi k/N$.

Two properties matter operationally.

Hermitian symmetry. For real-valued $x[n]$, $X[N-k] = X[k]^*$, so the magnitude spectrum is symmetric about $k = N/2$ and only the first $\lfloor N/2 \rfloor + 1$ bins carry independent information. This is why the code slices `fft_data[:N//2]` and why the library’s `RFFT` class exists at all.

Bin frequencies. Bin k corresponds to physical frequency

$$f_k = \frac{k f_s}{N}, \quad 0 \leq k < N/2,$$

which is exactly what `scipy.fftpack.fftfreq(N, 1/sample_rate)` returns.

4.2 Frequency resolution and the time–frequency trade-off

The spacing between adjacent DFT bins is

$$\Delta f = \frac{f_s}{N} = \frac{1}{NT_s} = \frac{1}{T_{\text{win}}}$$

where $T_{\text{win}} = NT_s$ is the window duration in seconds. **Frequency resolution is the reciprocal of observation time, and depends on nothing else.** Decimating does not degrade it: reducing

f_s by M also reduces N by M , so Δf is unchanged. This is a point worth emphasising, because it is frequently misunderstood: decimation buys a reduction in computational cost and costs nothing in frequency resolution.

For the deployed configuration:

Quantity	Raw (44.1 kHz)	Decimated (501 Hz)
Window duration	5 s	5 s
Samples N	220,500	2,506
Δf	0.200 Hz	0.200 Hz
Bins in [10, 200] Hz	950	950

A 0.200 Hz bin spacing translates, through the RPM formula of Section 7 with `EVENTS_PER_CYCLE` = 4, into an RPM quantisation of

$$\Delta\Omega = \Delta f \cdot \frac{60}{4} = 0.200 \times 15 = 3.0 \text{ rpm},$$

which is 0.3% at 1000 rpm and 0.05% at 5800 rpm. Quantisation of the frequency axis is therefore *not* a limiting error source for this application — a point that will matter in Section 12, where the observed scatter is an order of magnitude larger and must therefore have a different cause.

The trade-off runs the other way. Lengthening the window sharpens Δf but assumes the engine speed is constant across the window. During a throttle transient the true frequency sweeps, the spectral line smears across many bins, and the peak estimate degrades. Five seconds is a compromise: fine enough in frequency to be irrelevant as an error source, coarse enough in time that a rapid power change is averaged rather than resolved. Section 13 returns to the latency cost of this choice.

4.3 The FFT: Cooley–Tukey decimation in time

Evaluated directly, the DFT costs N^2 complex multiply–accumulates. For $N = 220,500$ that is 4.9×10^{10} operations — entirely infeasible for a five-second real-time budget on a Pi Zero. The fast Fourier transform reduces this to $\mathcal{O}(N \log N)$. The formulation in universal use descends from Cooley and Tukey’s 1965 paper [cooley1965]. It is worth being precise about what that paper contains: Cooley and Tukey present the *general composite- N* factorisation, recursively decomposing a DFT of size $N = N_1 N_2$ into transforms of sizes N_1 and N_2 , and observe that the total cost is proportional to $N \log N$ — under $2N \log_2 N$ operations when N is a power of two, which they note is the most favourable case on a binary machine. The radix-2 decimation-in-time algorithm derived below is the special case $N_1 = 2$ applied recursively, and is the form in which the idea is usually taught.

The radix-2 decimation-in-time derivation is short enough to give in full. Let N be even and split the sum by parity of n :

$$X[k] = \sum_{n \text{ even}} x[n] W_N^{kn} + \sum_{n \text{ odd}} x[n] W_N^{kn}.$$

Substituting $n = 2r$ and $n = 2r + 1$:

$$X[k] = \sum_{r=0}^{N/2-1} x[2r] W_N^{2rk} + W_N^k \sum_{r=0}^{N/2-1} x[2r+1] W_N^{2rk}.$$

The key identity is $W_N^2 = e^{-j4\pi/N} = e^{-j2\pi/(N/2)} = W_{N/2}$, giving

$$\boxed{X[k] = E[k] + W_N^k O[k]}$$

where $E[k]$ and $O[k]$ are the $N/2$ -point DFTs of the even- and odd-indexed subsequences. Because E and O are periodic with period $N/2$ and $W_N^{k+N/2} = -W_N^k$, the second half follows for free:

$$X[k + N/2] = E[k] - W_N^k O[k].$$

Each pair $(X[k], X[k + N/2])$ therefore costs one complex multiplication (by the *twiddle factor* W_N^k) and two complex additions — the *butterfly*. Recursing on the two half-length transforms until length 1, the cost recurrence is

$$T(N) = 2T(N/2) + \mathcal{O}(N) \implies T(N) = \mathcal{O}(N \log_2 N).$$

For $N = 220,500$ this is a speed-up of roughly $N/\log_2 N \approx 12,400$. Modern libraries generalise the same idea to mixed radices and prime factors, so the $\mathcal{O}(N \log N)$ behaviour is not restricted to powers of two — which matters here, since $2506 = 2 \times 7 \times 179$ and $220500 = 2^2 \times 3^2 \times 5^3 \times 7^2$ are not powers of two.

4.4 Spectral leakage and the need for a window

The DFT of a length- N record is implicitly the DTFT of the *infinite* sequence formed by periodically extending that record. Equivalently, the finite record is the infinite signal multiplied by a rectangular window

$$w_R[n] = \begin{cases} 1, & 0 \leq n \leq N-1 \\ 0, & \text{otherwise.} \end{cases}$$

Multiplication in time is convolution in frequency, so the observed spectrum is the true spectrum convolved with the window's transform, the Dirichlet kernel

$$W_R(e^{j\omega}) = e^{-j\omega(N-1)/2} \frac{\sin(\omega N/2)}{\sin(\omega/2)}.$$

If the analysed sinusoid completes an exact integer number of cycles in the window, the kernel's zeros land exactly on the other bin centres and the spectrum is a single clean line. In general it does not. The energy then *leaks* into all bins, falling off only as the Dirichlet sidelobes, whose first sidelobe is just -13.3 dB below the mainlobe and which decay at -6 dB per octave.

For this application the consequence is concrete. Engine harmonics are strong, closely spaced, and of widely differing amplitude. A -13 dB sidelobe skirt from a dominant order can bury or displace

a weaker neighbouring order twenty decibels down. Harris’s 1978 survey [harris1978] remains the standard catalogue of window functions and their trade-offs, and the trade-off is always the same: wider mainlobe (worse resolution) in exchange for lower sidelobes (less leakage).

The system uses the **Hann window** (called `np.hanning` in NumPy):

$$w[n] = \frac{1}{2} \left[1 - \cos\left(\frac{2\pi n}{N-1}\right) \right] = \sin^2\left(\frac{\pi n}{N-1}\right), \quad 0 \leq n \leq N-1.$$

Its properties, following Harris’s tabulation (Harris rounds the sidelobe figures to the nearest decibel; the exact values are given here and were re-measured numerically for this document), are:

Property	Rectangular	Hann
Mainlobe width (bins, -3 dB)	0.89	1.44
Highest sidelobe	-13.26 dB	-31.47 dB
Sidelobe falloff	-6 dB/oct	-18 dB/oct
Scalloping loss	3.92 dB	1.42 dB
Coherent gain	1.00	0.50

The Hann window earns its place here through its sidelobe behaviour. Measuring the sidelobe-peak envelope directly:

Offset from peak	Rectangular	Hann
1.43 bins	-13.26 dB	—
2.36–2.46 bins	-17.83 dB	-31.47 dB
3.4 bins	-20.79 dB	-41.48 dB
4.4 bins	-22.99 dB	-48.48 dB
5.5 bins	-24.74 dB	-53.94 dB

Five bins from a dominant order, the rectangular window still leaks at -25 dB while the Hann window has fallen to -54 dB — a 29 dB improvement in exactly the region where neighbouring engine orders live. Since the orders in this system’s measured spectra span roughly 28 dB in amplitude (Section 7), rectangular leakage from the strongest order would be comparable to the weakest orders themselves, whereas Hann leakage is negligible against them.

A caution is required about the *other* commonly cited advantage, scalloping loss, because it does not rescue this system. Scalloping loss is the amplitude under-reading suffered when a tone falls midway between bin centres: worst case 3.92 dB for a rectangular window and 1.42 dB for Hann. Section 12 shows that the two competing harmonics in this system’s real data are separated by only 0.23 dB. **That margin is smaller than the worst-case scalloping loss of either window, including Hann.** Bin-grid alignment alone can therefore invert the measured ranking of the two orders, and the Hann window reduces but does not eliminate the effect. This is an additional, window-independent contributor to the order-flip failures measured in Section 12, and it can only be removed by interpolating the peak (Section 14), not by a better window.

The implementation applies the window directly:

```

windowed_signal = signal * np.hanning(len(signal))
N = len(windowed_signal)
fft_data = np.abs(fftpack.fft(windowed_signal)[:N//2])
freqs = fftpack.fftfreq(N, 1/sample_rate)[:N//2]

```

Two observations. First, the transform is a *full complex* FFT whose upper half is then discarded; a real-input transform (`numpy.fft.rfft`, or the library’s `RFFT` class) would give the same result at roughly half the cost, and this is a straightforward available optimisation. Second, no amplitude correction for the window’s coherent gain of 0.5 is applied. That is harmless here, because the algorithm uses only the *location* of the maximum, and a constant scale factor cannot move an `argmax` — but it would need fixing if absolute levels were ever reported.

4.5 The STFT and time-varying spectra

A single DFT assumes stationarity over the whole record. Engine speed is not stationary over a flight. The standard generalisation is the *short-time Fourier transform*: partition the signal into (usually overlapping) segments, window each, and transform each,

$$X[m, k] = \sum_{n=0}^{L-1} w[n] x[n + mH] e^{-j2\pi kn/L},$$

where L is the segment length and H the hop. The squared magnitude $|X[m, k]|^2$ is the *spectrogram*, whose engine-speed signature is a family of bright, parallel, upward- and downward-sweeping ridges — the orders. The resolution trade-off is now explicit and local: $\Delta f = f_s/L$ against $\Delta t = H/f_s$.

The deployed system implements a *degenerate STFT*: the recorder emits non-overlapping five-second segments ($L = N$, $H = N$, zero overlap) and the analyser transforms each independently. There is no cross-segment state at all. The research library’s STFT class in `audio_src/FeatureEngineering/Spectrograms/Fourier.py` wraps `scipy.signal.stft` with a proper Hann window and configurable overlap, and would be the natural substrate for the ridge-tracking estimator proposed in Section 14.

5 Filtering and Decimation Theory

5.1 Why a filter must precede downsampling

Downsampling by an integer factor M retains every M -th sample:

$$x_d[m] = x[mM].$$

In the frequency domain this is again spectral replication, but now with the replicas spaced by the *new*, lower sampling frequency f_s/M :

$$X_d(e^{j\omega}) = \frac{1}{M} \sum_{i=0}^{M-1} X(e^{j(\omega-2\pi i)/M}).$$

Everything in the original signal above the new Nyquist frequency $f_s/(2M)$ folds into the new baseband. The condition for this to be harmless is exactly the Nyquist criterion restated at the new rate:

$$X(e^{j\omega}) = 0 \quad \text{for } |\omega| \geq \pi/M$$

which in physical units is $|f| \geq f_s/(2M)$.

Raw microphone audio manifestly does not satisfy this. It contains energy across the full audible band. **Downsampling must therefore be preceded by a lowpass filter with cut-off at or below $f_s/(2M)$.** The filter-then-downsample cascade is *decimation*, and the theory is treated comprehensively in Crochiere and Rabiner's tutorial review [crochiere1981] and monograph [crochiere1983].

5.2 The Butterworth response

Butterworth's 1930 paper [butterworth1930] is the origin of the response now universally named after him. His stated design goal, in his own words, was that "an ideal electrical filter should not only completely reject the unwanted frequencies but should also have uniform sensitivity for the wanted frequencies", and the paper's aim is "to obtain a filter factor F , that is, the ratio of the output e.m.f. to the input e.m.f., of the form $F = (1 + x^m)^{-1}$ ", where $x = f/f_0$ and " m increases with the number of elements employed". His F is the *squared* magnitude ratio: for the single-element (second-order) section he derives $(E_1/E_2)^2 = 1 + x^4$, hence $F = (1 + x^4)^{-1}$, and for a cascade of n intervalve elements $F = (1 + x^{2n})^{-1}$.

In modern notation, an order- n Butterworth lowpass with cutoff ω_c has

$$|H(j\omega)|^2 = \frac{1}{1 + \left(\frac{\omega}{\omega_c}\right)^{2n}}$$

which is exactly Butterworth's F with $m = 2n$.

The *maximally flat* characterisation — that the first $2n - 1$ derivatives of $|H|^2$ with respect to ω vanish at $\omega = 0$, so the passband is as flat as an order- n rational function permits — is the modern restatement of what Butterworth called “uniform sensitivity in the pass region”; the derivative formulation is later terminology, not his. It follows directly from the series $|H|^2 = 1 - u^{2n} + u^{4n} - \dots$ with $u = \omega/\omega_c$, whose first non-vanishing derivative at the origin is of order $2n$.

At cutoff, $|H| = 1/\sqrt{2}$ (−3 dB) for every order — Butterworth’s own $F = 1/2$ at $x = 1$. In the stopband, $|H| \rightarrow (\omega_c/\omega)^n$, so the asymptotic roll-off is $-20n$ dB/decade, or $-6n$ dB/octave.

The pole locations are standard later filter theory rather than a result of the 1930 paper, which works directly with element values. The n poles lie on a semicircle of radius ω_c in the left half-plane at

$$s_k = \omega_c e^{j\pi(2k+n-1)/(2n)}, \quad k = 1, \dots, n,$$

an arrangement that is numerically well-behaved and is part of why Butterworth designs are the default choice throughout the research library [oppenheim2010].

The library’s `audio_src/FeatureEngineering/Filters/Butterworth.py` provides `LowPassFilter`, `HighPassFilter`, `BandPassFilter`, and `BandStopFilter`. The bandpass implementation is the one used by the adaptive tracker of Section 8:

```
nyquist = sample_rate / 2
low = lowcut / nyquist
high = highcut / nyquist
b, a = butter(order, [low, high], btype='band', analog=analog, output='ba')
...
y = filtfilt(b, a, data)
```

Two design points are worth naming. Normalisation is by the Nyquist frequency, which is SciPy’s convention. And filtering is applied with `filtfilt`, which runs the filter forwards and then backwards over the record. This makes the effective response $|H(j\omega)|^2$ — exactly zero phase distortion, at the cost of doubled attenuation in dB and non-causality. For offline or block processing this is strictly desirable: an IIR filter’s nonlinear phase would otherwise smear the relative timing of the combustion impulses, which is precisely the structure being measured.

A defect worth recording. The `LowPassFilter` and `HighPassFilter` classes in the same file normalise with `nyquist = sample_rate` rather than `sample_rate / 2`. The resulting normalised cutoff is half its intended value, so a filter requested at 200 Hz is actually realised at 100 Hz. `BandPassFilter` and `BandStopFilter` use the correct `sample_rate / 2`. Since the deployed pipeline uses neither of the two affected classes, this defect has no effect on any result in this document, but it should be corrected before those classes are used.

5.3 The decimation actually performed

`rpi_test_final.py` decimates in the Decimation block:

```
decimation_factor = int(original_rate // self.target_rate)
if decimation_factor < 1:
    decimated_signal = signal
```

```

    decimated_rate = original_rate
else:
    decimated_signal = decimate(signal, decimation_factor, axis=0, ftype='iir')
    decimated_rate = original_rate // decimation_factor

```

With `SAMPLE_RATE = 44100` and `DECIMATED_RATE = 500`:

$$M = \left\lfloor \frac{44100}{500} \right\rfloor = 88, \quad f_s^{(d),\text{true}} = \frac{44100}{88} = 501.136 \text{ Hz}, \quad f_s^{(d),\text{recorded}} = \left\lfloor \frac{44100}{88} \right\rfloor = 501 \text{ Hz}.$$

Note that the code records the *floor* of the true decimated rate. The relative error is

$$\frac{501.136 - 501}{501.136} = 2.72 \times 10^{-4} = 0.027\%,$$

which propagates directly into the frequency and hence RPM estimates as a systematic -0.027% bias — about -0.27 rpm at 1000 rpm, -1.6 rpm at 5800 rpm. This is negligible against every other error source identified in this document, but it is a genuine bias rather than noise, and it is trivially removable by carrying the rate as a float. It is worth noting that the simulation audio in the repository is sampled at 48,000 Hz, for which $M = 96$ divides exactly and the bias vanishes entirely — so this particular defect is invisible in Level-1 testing and appears only on the deployed hardware path.

5.4 The anti-aliasing filter that SciPy actually uses

Despite the surrounding library being built around Butterworth designs, the deployed decimation does not use one. `scipy.signal.decimate` with `ftype='iir'` and no explicit order uses, per its documentation and source, an **order-8 Chebyshev type I filter with 0.05 dB passband ripple and normalised passband edge $0.8/M$** , applied with `zero_phase=True` (that is, via `filtfilt`) [scipydecimate]. In SciPy 1.13 the design line is literally `sos = cheby1(n, 0.05, 0.8 / q, output='sos')`.

Chebyshev type I trades equiripple passband error for a much steeper transition than Butterworth at equal order — the right trade when the passband tolerance is 0.05 dB and the transition band is narrow, as it is here.

The normalised edge $0.8/M$ places the physical passband edge at

$$f_{\text{pass}} = \frac{0.8}{M} \cdot \frac{f_s}{2} = 0.4 \frac{f_s}{M} = 0.4 f_s^{(d)},$$

which for $M = 88$, $f_s = 44100$ Hz gives $f_{\text{pass}} = 200.45$ Hz, and for $M = 96$, $f_s = 48000$ Hz gives exactly 200.00 Hz.

This is a striking and fortunate coincidence: the anti-aliasing filter’s passband edge lands on 200 Hz, precisely the upper edge of the peak finder’s [10, 200] Hz search band.

The two constants were chosen independently — 200 Hz from engine harmonics, $0.4 f_s^{(d)}$ from a SciPy default — and they agree. The analysis band is exactly the filter’s passband, with the entire 200–250 Hz guard band available for roll-off.

Measuring the realised response of `cheby1(8, 0.05, 0.8/88)` at $f_s = 44100$ Hz, and doubling the decibel figures to account for `filtfilt`:

Frequency	Magnitude (single pass)	Effective (<code>filtfilt</code>)
10 Hz	−0.043 dB	−0.085 dB
33 Hz	−0.003 dB	−0.006 dB
67 Hz	−0.042 dB	−0.084 dB
150 Hz	−0.039 dB	−0.079 dB
200 Hz	−0.037 dB	−0.073 dB
210 Hz	−1.475 dB	−2.951 dB
250.6 Hz (new Nyquist)	−22.80 dB	−45.59 dB
300.7 Hz	−41.51 dB	−83.02 dB
501.1 Hz ($= f_s^{(d)}$)	−83.52 dB	−167.03 dB

Across the entire $[10, 200]$ Hz analysis band the total deviation is under 0.1 dB — the band is genuinely flat, so the relative amplitudes of the engine orders are preserved and the peak-picking comparison between them is not biased by the filter.

The aliasing figure can be stated exactly. A component at continuous frequency f folds into the decimated band $[10, 200]$ Hz if and only if

$$f \in [if_s^{(d)} + 10, if_s^{(d)} + 200] \cup [if_s^{(d)} - 200, if_s^{(d)} - 10], \quad i = 1, 2, \dots$$

The nearest such interval is $[301.1, 491.1]$ Hz, over which the effective attenuation is at least 83 dB and rises rapidly. Every higher image is attenuated by more than 167 dB. **Aliasing contamination of the analysis band is therefore suppressed by at least 83 dB, which is far below the noise floor of any realistic cabin recording, and can be discounted entirely as an error source.**

5.5 The decimation factor is out of specification

One warning must be recorded. SciPy’s documentation states explicitly: “*When using IIR down-sampling, it is recommended to call `decimate` multiple times for downsampling factors higher than 13.*” [`scipydecimate`]. The deployed factor is $M = 88$, nearly seven times that recommendation.

The reason for the recommendation is numerical conditioning: at $M = 88$ the normalised passband edge is $0.8/88 = 0.00909$, so all eight poles are crowded into a tiny arc very close to $z = 1$ on the unit circle. In direct-form transfer-function arithmetic this is badly conditioned, and evaluating the same design as `(b, a)` coefficients rather than second-order sections produces visible numerical artefacts — a spurious +2.3 dB passband error was observed in exactly this test. SciPy internally uses the `output='sos'` cascade, which is numerically robust, and the measured SOS response tabulated above is clean to within the design’s 0.05 dB ripple specification.

So the deployed configuration is *empirically* sound, and this document’s own measurements confirm it. But it is sound by virtue of an implementation detail of the library rather than by design, and it is fragile to a library change. The correct engineering practice is a multi-stage cascade, for example $88 = 11 \times 8$ or $88 = 4 \times 22$, or better a three-stage $2 \times 4 \times 11$, which would also be *cheaper*:

filtering at progressively reduced rates means fewer multiply-accumulates per input sample. This is recommended in Section 14.

6 Approaches to Rotating-Machinery Frequency and Order Estimation

Before analysing this system’s estimator in detail, it is useful to place it in the landscape of established techniques. The literature on extracting rotational information from vibration and acoustic signals is mature; Randall’s monograph [randall2011] is the standard reference for the machine-condition-monitoring perspective.

6.1 Order analysis: the organising idea

The organising concept for all rotating machinery is the *order*. Given a shaft rotating at Ω rpm, the shaft frequency is $f_1 = \Omega/60$ Hz, and a spectral component at frequency f is said to lie at order

$$o = \frac{f}{f_1} = \frac{60f}{\Omega}.$$

Orders are dimensionless and, crucially, *invariant to speed*. A component at order 2 stays at order 2 whether the engine is idling or at full power, even though its absolute frequency doubles. Mechanical phenomena are naturally described in orders — firing at order 2 for a four-cylinder four-stroke, blade-pass at order B for a B -bladed propeller, and gear mesh at the tooth count — while measurement is naturally performed in Hz. Order analysis is the machinery for moving between the two.

6.2 FFT peak-picking

The simplest estimator, and the one this system implements, computes a magnitude spectrum and takes the argmax over a band of interest:

$$\hat{f}_{\text{peak}} = \arg \max_{f \in [f_{\min}, f_{\max}]} |X(f)|, \quad \hat{\Omega} = \frac{60 \hat{f}_{\text{peak}}}{o_{\text{assumed}}}.$$

Its virtues are real and should not be understated: it is $\mathcal{O}(N \log N)$, it has no state, no tuning parameters beyond the band edges, no convergence behaviour, no failure modes involving divergence, and it can be verified by inspection. On a Raspberry Pi Zero these are decisive advantages.

Its theoretical accuracy for an isolated tone in white noise is excellent. Rife and Boorstyn [rife1974] derived the Cramer–Rao lower bound for single-tone frequency estimation from N samples at SNR γ :

$$\text{var}(\hat{f}) \geq \frac{6f_s^2}{(2\pi)^2 \gamma N(N^2 - 1)},$$

showing that the variance falls as N^{-3} . Two qualifications matter, and both are made in the original paper.

First, the estimator that attains this bound is the *maximiser of the periodogram* over a continuum of frequencies — the maximum-likelihood estimator. The DFT peak is a coarse, bin-quantised

approximation to it. The gap is closed by interpolating the peak (parabolic or Jacobsen interpolation on the three bins around the maximum), which typically recovers an order of magnitude in resolution at negligible cost; without interpolation the estimate is floor-limited by Δf .

Second — and this is the qualification that matters most here — Rife and Boorstyn identify a **threshold effect**. Above a threshold SNR the estimator tracks the bound; below it, the variance departs from the bound abruptly and catastrophically, because the global maximum of the periodogram starts landing on the *wrong* peak. The error is then not a small perturbation of the correct frequency but a gross outlier at a different frequency altogether.

The assumption underlying both the bound and the threshold analysis is *isolation*: a single tone in white noise. An engine spectrum is the opposite — a dozen comparable harmonics of a common fundamental — and so the argmax reports which harmonic is largest, a quantity that depends on exhaust geometry, microphone placement, load, and speed, and that is not stable even within one recording. The resulting failures are structurally the same as Rife and Boorstyn’s threshold outliers, but they are driven by competing *deterministic* components rather than by noise, so they do not disappear at high SNR. Section 12 measures exactly this instability.

6.3 Order tracking and the Vold–Kalman filter

The classical fix is to make the analysis *angle-referenced* rather than *time-referenced*. If a tachometer or encoder provides shaft angle $\theta(t)$, one resamples the signal at uniform increments of θ rather than of t . In the angle domain every order becomes a fixed “frequency” in cycles-per-revolution regardless of speed, so orders that would smear across dozens of bins during a run-up collapse to sharp lines.

Vold and Leuridan’s Vold–Kalman order tracking filter [vold1995] is the most influential development of this idea. Rather than resampling, it poses order extraction as a state-estimation problem: each order is modelled as a complex envelope modulating a known carrier phase derived from the measured speed, and a non-stationary Kalman-style filter estimates the envelopes. Two properties make it powerful: it has effectively unlimited frequency resolution, being limited by envelope smoothness rather than window length; and *coupled* formulations can separate orders that cross or nearly coincide, which no fixed filter bank can do. It handles extreme slew rates — run-ups and shutdowns — where conventional tracking filters fail outright.

The barrier for this project is that Vold–Kalman needs a phase reference, and the whole point of the system is that there is no tachometer.

6.4 Tacholeless order tracking

The subfield addressing exactly that barrier is *tacholeless order tracking*, in which the speed profile is estimated from the vibration or acoustic signal itself and then used as if it came from a tachometer. Lu and colleagues [lu2019] review the field comprehensively; Peeters and colleagues [peeters2019] compare the leading methods head-to-head on common experimental data.

The dominant family works by *ridge tracking* on a time-frequency representation: compute an STFT or wavelet transform, identify the trajectory of a chosen order as a connected ridge of maxima, and integrate the instantaneous frequency to recover phase. The refinements that make it robust are instructive for the present work:

- **Multi-order fusion.** Rather than tracking one ridge, combine evidence from several harmonically related ridges. This is the essential insight, because the ambiguity that defeats single-peak picking is resolved by the *pattern* of components, not by any one of them.
- **Kinematic constraints.** Physical shafts have bounded angular acceleration, so a candidate trajectory implying an implausible slew can be rejected. A probabilistic formulation incorporating angular-acceleration priors into maxima tracking is now standard.
- **Iterative refinement.** A coarse profile is used to resample, the sharper angle-domain spectrum yields a better profile, and the process is repeated.

Section 14 proposes exactly this family as the successor to the current estimator, and Section 12 gives a measured demonstration that the multi-order idea alone already resolves this system’s dominant failure mode.

6.5 Envelope and cepstral analysis

Two further techniques deserve mention for completeness, since they address *diagnosis* rather than speed estimation and are the natural extension once speed tracking is solved.

Envelope (demodulation) analysis is the standard method for localised faults in rolling-element bearings [randallantoni2011]. A bearing defect produces a train of short impacts that excites high-frequency structural resonances; the resonance itself carries no diagnostic information, but its *amplitude modulation* repeats at a characteristic defect frequency. Bandpass filtering around the resonance, rectifying via the Hilbert transform, and spectrally analysing the envelope reveals the defect frequency directly.

Cepstral analysis transforms the log spectrum back to a pseudo-time (“quefrency”) domain, where a uniformly spaced harmonic family collapses to a single peak at the reciprocal of its spacing. Since the difficulty in this system is precisely determining the *spacing* of a harmonic family rather than the position of any one member, the real cepstrum is a directly relevant tool, and Section 14 proposes it as a cross-check on the harmonic-template estimator.

Underlying both is the observation that rotating-machine signals are *cyclostationary* — their statistical moments, not merely their waveforms, are periodic in shaft angle. Antoni’s tutorial [antoni2009] develops this framework and explains why second-order cyclostationary tools such as the spectral correlation density are the theoretically correct home for these signals.

6.6 Where this system sits

Method	Needs tachometer	Cost	Resolves order ambiguity	Used here
FFT peak-picking	No	Very low	No	Yes
Angle resampling	Yes	Low	Yes	No
Vold–Kalman	Yes	Medium	Yes	No
Tachless ridge tracking	No	Medium	Yes	Proposed
Harmonic template / cepstrum	No	Low–medium	Yes	Proposed
Envelope analysis	No	Medium	N/A (diagnosis)	No

The system occupies the cheapest cell in the table and pays for it in the one column that matters most. Sections 12 and 13 quantify that payment; Section 14 proposes moving one row down, to

harmonic-template estimation, which is measured in Section 12 to remove the dominant failure at a cost the Pi can afford.

7 Harmonic Analysis and the RPM Formula

This section derives the conversion from a measured acoustic frequency to crankshaft speed. It is the conceptual centre of the system, and it is also where the system's principal assumption is made.

7.1 Kinematics of the four-stroke cycle

In a four-stroke engine each cylinder executes intake, compression, power, and exhaust in sequence. Each stroke corresponds to half a crankshaft revolution, so one complete thermodynamic cycle occupies

$$4 \text{ strokes} \times \frac{1}{2} \text{ rev/stroke} = 2 \text{ crankshaft revolutions.}$$

Each individual cylinder therefore fires once every two crankshaft revolutions. This factor of two is the defining kinematic feature of the four-stroke cycle and the origin of *half-order* components in its spectrum.

For an engine of C cylinders with evenly distributed firing, the number of combustion events per crankshaft revolution is

$$E = \frac{C}{2}.$$

For the Rotax 912's $C = 4$:

$$E = \frac{4}{2} = 2 \text{ firing events per revolution.}$$

The firing (combustion) frequency at crankshaft speed Ω rpm is therefore

$$f_{\text{fire}} = E \cdot \frac{\Omega}{60} = \frac{2\Omega}{60} = \frac{\Omega}{30} \text{ Hz}$$

and the firing event sits at **order 2**.

The full order structure of a four-cylinder four-stroke follows immediately. Because the cycle repeats every two revolutions, the fundamental period of the whole process is $2 \times 60/\Omega$ seconds, giving a fundamental frequency at **order 0.5**. Every physically realisable component is therefore an integer multiple of order 0.5:

$$o \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0, 3.5, 4.0, \dots\}.$$

Among these:

- **Order 0.5** — the complete-cycle fundamental; also the camshaft rate, since the camshaft turns at half crankshaft speed.
- **Order 1** — shaft rotation; reciprocating unbalance, misalignment, propeller once-per-rev.
- **Order 2** — the firing frequency, and also the second-order reciprocating inertia force inherent to a piston-crank mechanism.

- **Order 4, 6, 8** — harmonics of firing. The combustion pressure pulse is a sharp, non-sinusoidal impulse, so the firing event necessarily distributes substantial energy into its own harmonics.
- **Odd half-orders (1.5, 2.5, 3.5, ...)** — these appear when cylinders are not perfectly matched. Any cylinder-to-cylinder variation in charge, compression, or ignition timing breaks the exact one-revolution symmetry and modulates the firing train at the two-revolution cycle rate, generating sidebands spaced at order 0.5 around the firing harmonics. Their presence and relative strength is a classical indicator of cylinder imbalance.

7.2 Deriving the RPM formula

The system’s conversion is

```
rpm = (peak_freq * 60) / self.events_per_cycle
```

Derived from first principles. Let $\hat{f}_{\text{peak}}$ be the measured frequency in Hz — that is, cycles per second — of a component known to occur ν times per crankshaft revolution. Then:

Events per second: $\hat{f}_{\text{peak}}$ (by definition of Hz).

Revolutions per second:

$$\frac{\hat{f}_{\text{peak}} \text{ events/s}}{\nu \text{ events/rev}} = \frac{\hat{f}_{\text{peak}}}{\nu} \text{ rev/s.}$$

Revolutions per minute:

$$\Omega = \frac{\hat{f}_{\text{peak}}}{\nu} \times 60 = \frac{60 \hat{f}_{\text{peak}}}{\nu} \text{ rpm}$$

which is exactly the code, with $\nu = \text{EVENTS_PER_CYCLE}$. The dimensional check is clean: $[\text{s}^{-1}] \times [\text{s} \cdot \text{min}^{-1}] / [\text{rev}^{-1}] = [\text{rev} \cdot \text{min}^{-1}]$.

The parameter ν is not a free constant. It is *the order of the spectral component that the peak-picker selected*, and the estimate is correct if and only if that order is what ν says it is.

7.3 Why EVENTS_PER_CYCLE = 4, not 2

Kinematics gives $E = 2$ for a four-cylinder four-stroke. The earliest iteration in the repository, `rpi_test.py`, encodes exactly that:

```
EVENTS_PER_CYCLE = 2
```

as does `rpi_test_fft.py`, and as does the `PistonEngineConfig` class in `rpi_test_new.py`, whose comment reads `self.events_per_cycle = 2 # 4-stroke engine`. The deployed `rpi_test_final.py` changed it:

```
EVENTS_PER_CYCLE = 4
```

The README explains the change as follows: “*EVENTS_PER_CYCLE = 4 is set to match the 2nd harmonic of the Rotax 912 firing frequency, which dominates in real recordings.*”

The arithmetic is consistent. If firing is order 2 and the *second harmonic of firing* is what the peak-picker selects, then that component sits at

$$o = 2 \times 2 = 4,$$

so $\nu = 4$ and

$$\Omega = \frac{60\hat{f}_{\text{peak}}}{4} = 15\hat{f}_{\text{peak}}.$$

It is important to be precise about what this constant *is* and *is not*. It is not a claim that four combustion events occur per revolution — that would be physically false for a four-stroke. It is an *empirical order index*: an assertion that the loudest component in $[10, 200]$ Hz is reliably the one at order 4.

7.4 Verifying the order assignment against measured data

That assertion is testable, and this document tests it. The repository’s reference recording `simulation_data/audios/1000_rpm.wav` is a Rotax 912 recording labelled 1000 rpm. Running the repository’s own decimation on the whole file (48,000 Hz source, $M = 96$, exact 500 Hz output, $N_d = 36,822$, $\Delta f = 0.0136$ Hz) and measuring the amplitude at each order of a half-order family based at $f_{0.5} = 8.368$ Hz gives:

Order o	Frequency (Hz)	Relative amplitude
0.5	8.368	0.039
1.0	16.736	0.041
1.5	25.104	0.101
2.0	33.472	0.974
2.5	41.840	0.240
3.0	50.208	0.073
3.5	58.576	0.435
4.0	66.944	1.000
4.5	75.312	0.297
5.0	83.680	0.103
5.5	92.048	0.087
6.0	100.416	0.424
7.5	125.520	0.094
8.0	133.888	0.228

This is a textbook four-cylinder four-stroke order spectrum, and it confirms the order model in three independent ways.

The family is a half-order family. Every significant component is an integer multiple of 8.368 Hz. That spacing corresponds to the two-revolution cycle fundamental of a four-stroke, exactly as predicted.

The even orders dominate. Orders 2, 4, 6, and 8 carry relative amplitudes 0.974, 1.000, 0.424, and 0.228, while orders 1, 3, and 5 carry only 0.041, 0.073, and 0.103. This is the signature of a firing harmonic series built on order 2 — and it discriminates decisively against the alternative

hypothesis that the shaft is at half this speed, which would require the *odd* orders 3, 5, 7 to be the strong ones.

The implied speed matches the label. Taking order 0.5 as $f_{0.5} = 8.368$ Hz,

$$\Omega = 2 \times 60 \times f_{0.5} = 120 \times 8.368 = 1004.2 \text{ rpm},$$

against a file labelled 1000 rpm — an agreement of 0.4%. Equivalently, from the firing component at order 2: $\Omega = 30 \times 33.472 = 1004.2$ rpm. Equivalently from order 4 with $\nu = 4$: $\Omega = 15 \times 66.944 = 1004.2$ rpm. All three routes agree.

The EVENTS_PER_CYCLE = 4 calibration is therefore correct for this recording, and correct for a defensible physical reason: order 4 is genuinely the largest component in the analysis band.

(For completeness: 1004 rpm is below the Rotax 912's normal idle, so this recording almost certainly captures a start-up, cranking, or shut-down condition rather than a normal ground idle. That does not affect the order analysis, which is speed-invariant by construction.)

7.5 The margin, and why it is the system's central risk

The decisive number in the table above is not that order 4 wins. It is *by how much*. Order 4 has relative amplitude 1.000; order 2 has 0.974. The margin is

$$20 \log_{10} \left(\frac{1.000}{0.974} \right) = 0.23 \text{ dB}.$$

The entire correctness of the RPM output rests on a 0.23 dB amplitude difference. Any perturbation exceeding a quarter of a decibel — a change in microphone position, a different exhaust back-pressure, a partly open cowling, a change in engine load, or simply a different five-second slice of the same recording — can invert the ordering. When it does, the peak-picker returns 33.47 Hz instead of 66.94 Hz, and the formula with $\nu = 4$ reports

$$\Omega = 15 \times 33.47 = 502 \text{ rpm}$$

instead of 1004 — a 50% error, reported with no indication that anything has gone wrong.

This is not a hypothetical. Section 12 measures the flip rate directly, on this very recording, and finds it to be 21%.

8 Peak Detection and Frequency Tracking

8.1 The deployed peak finder

The whole estimator is `FrequencyPeakFinder.extract_features` in `rpi_test_final.py`. Stripped of error handling, its core is:

```
windowed_signal = signal * np.hanning(len(signal))
N = len(windowed_signal)
fft_data = np.abs(fftpack.fft(windowed_signal)[:N//2])
freqs = fftpack.fftfreq(N, 1/sample_rate)[:N//2]
valid_range = (freqs >= 10) & (freqs <= 200)
if np.any(valid_range):
    valid_fft = fft_data[valid_range]
    valid_freqs = freqs[valid_range]
    if np.max(valid_fft) > 0:
        peak_freq = valid_freqs[np.argmax(valid_fft)]
    else:
        peak_freq = 0.0
else:
    peak_freq = 0.0
```

Step by step:

1. **Window.** Hann, for the leakage and scalloping reasons of Section 4.
2. **Transform.** Full complex FFT, upper (redundant) half discarded.
3. **Magnitude.** $|X[k]|$; phase is unused.
4. **Band mask.** A boolean mask selects $10 \leq f \leq 200$ Hz. The band edges are *literals in the function body*, not module-level constants — a maintainability defect, since the same two numbers also implicitly govern the decimation design.
5. **Argmax.** The single largest bin in the masked band.
6. **Zero guard.** If the maximum masked magnitude is not strictly positive, the result is 0.0 Hz, which propagates to 0 rpm and engine status OFF. This is the path taken by a digitally silent input.

The properties of this estimator are worth stating plainly. It is **stateless** — no memory of previous chunks, so a spurious result is not propagated but neither is a correct one reinforced. It is **unsmoothed** — no median filter, no exponential average, no outlier rejection across chunks. It performs **no interpolation** — the estimate is quantised to the bin grid, though as Section 4 showed this contributes only 3 rpm and is not a limiting error. And it is a **hard argmax over a single band**, so it produces no confidence measure, no runner-up, and no indication that two candidates were within a quarter of a decibel of each other.

8.2 The duplicated raw-signal analysis

`FrequencyPeakFinder` actually runs the whole procedure twice: once on the *raw* 44,100 Hz signal, storing the result under `FREQUENCY_PEAK`, and once on the *decimated* signal, storing it under `DECIMATED_FREQUENCY_PEAK`.

The RPM block consumes the decimated result:

```
peak_freq = datum.get_derived_data(DerivedDataKey.DECIMATED_FREQUENCY_PEAK)
```

but the UART packet builder transmits the *raw* result:

```
peak_freq = datum.get_derived_data(DerivedDataKey.FREQUENCY_PEAK) or 0.0
```

The frequency transmitted to the ESP32 is therefore not the frequency from which the transmitted RPM was computed. In practice the two agree closely, because both analyse the same underlying signal over the same band — measured differences were 0.012 Hz on `1000_rpm.wav` (66.9585 vs 66.9708) and 0.004 Hz on `vo1o1.wav` (84.2411 vs 84.2373). So the discrepancy is benign numerically. But it is a genuine inconsistency, it defeats any downstream attempt to verify RPM against frequency, and it means the ESP32 cannot recompute or sanity-check the estimate it is given.

The raw-signal analysis is also the system’s single largest computational expense — a 220,500-point FFT computed on every chunk purely to populate a telemetry field — and eliminating it would recover essentially all of the savings that decimation was introduced to achieve. This is the highest-value, lowest-risk optimisation available, and it is recommended in Section 14.

8.3 The RPM block and its guards

```
rpm = (peak_freq * 60) / self.events_per_cycle
if rpm < 0 or rpm > 10000:
    rpm = 0.0
datum.set_derived_data(DerivedDataKey.RPM, rpm)
engine_status = 1 if rpm > 100 else 0
```

The upper clamp at 10,000 rpm is unreachable by construction: the band maximum is 200 Hz, so with $\nu = 4$ the maximum expressible RPM is $200 \times 15 = 3000$. Even with $\nu = 2$ it would be 6000. The guard is therefore dead code — harmless, but it gives a false impression of range checking that is not actually occurring. An effective guard would be the engine’s own envelope, roughly [400, 6000] rpm for a Rotax 912, and would catch the order-flip failures of Section 12 rather than passing them through silently.

Note also the interaction with ν : the reachable RPM ceiling of 3000 rpm means the deployed system *structurally cannot report* a Rotax 912 at cruise (4800–5500 rpm) or take-off (5800 rpm) power. Any in-flight measurement above about 3000 rpm must be reported as some lower harmonic-misassigned value. This is examined against real in-flight data in Section 12.

8.4 The adaptive TunableFilter

The research library takes a different approach in `audio_src/FeatureEngineering/Filters/Other.py`. `TunableFilter` maintains a bandpass filter whose centre frequency follows the detected peak, so that once the engine frequency is acquired the filter narrows around it and suppresses everything else. Its state is `(peak_freq, bandwidth, order)`, initialised in `EngineAnalysis.FrequencyPeakFinder` as `TunableFilter(init_freq=100, bandwidth=10, order=4)`.

Retuning is guarded against runaway:

```

if self.__peak_freq - self.__bandwidth / 2 < 20:
    self.__bp_filter.set_lowcut(10)
else:
    self.__bp_filter.set_lowcut(max(freq - self.__bandwidth / 2, 20))

if self.__peak_freq + self.__bandwidth / 2 > 150:
    self.__bp_filter.set_highcut(150)
else:
    self.__bp_filter.set_highcut(min(freq + self.__bandwidth / 2, 150))

```

so the passband is always contained in [10,150] Hz — note the 150 Hz ceiling, inconsistent with the 200 Hz ceiling used everywhere else in the system.

The filter is applied to a *rolling buffer* rather than to isolated chunks. The containing `FrequencyPeakFinder` maintains ten seconds of history:

```

self.__buffer = np.concatenate((self.__buffer, signal_array), axis=None)
self.__buffer = self.__buffer[-int(sample_rate * self.__buffer_seconds):]

```

This is a genuine architectural improvement over the deployed script. Ten seconds at 500 Hz gives 5,000 samples and $\Delta f = 0.1$ Hz, and the overlap between successive analyses means consecutive estimates are correlated, which suppresses chunk-to-chunk jitter — exactly the pathology measured in Section 12.

However, the peak selection inside `TunableFilter.apply_filter` has two defects that would prevent it working as intended:

```

peak_indices = scipy.signal.find_peaks(power_spectrum, distance=sample_rate)[0]
...
peak_index = peak_indices.min()
peak_frequency = self.__fft.get_freq()[peak_index]
self.set_peak_freq(peak_frequency)

```

First, `distance=sample_rate` sets the minimum peak separation to `sample_rate` *samples of the spectrum array*. At $f_s^{(d)} = 500$ Hz and a 5,000-point buffer, $\Delta f = 0.1$ Hz, so 500 array elements corresponds to a 50 Hz minimum separation — almost certainly not what was intended, and coarse enough to merge adjacent orders.

Second, and more seriously, `peak_indices.min()` selects the peak with the *lowest array index*, which is the *lowest-frequency* peak found — not the largest. The tracker therefore locks onto whatever low-frequency content survives the bandpass, rather than onto the dominant engine order.

There is an irony worth noting, though it should not be overstated. A *deliberate* lowest-frequency selection, with an amplitude threshold and a sensible separation, would be closer to correct than the `argmax` used in the deployed script, because the lowest member of a harmonic family is its fundamental and the fundamental fixes the speed unambiguously. That instinct points in the same direction as the harmonic-template estimator of Section 14 — though it is not the same algorithm, since it uses one component rather than scoring the whole comb. As written, however, with a 50 Hz effective separation and no amplitude threshold, it would latch onto noise. `TunableFilter` is not integrated into `rpi_test_final.py`, so none of this affects deployed behaviour; it is documented here so that the defects are fixed before integration.

9 System and Software Architecture

9.1 Why concurrency is required

Consider the naive sequential loop:

```
while True:
    audio = record(5 seconds)      # 5.00 s
    result = analyse(audio)        # T_a
    send_over_uart(result)         # T_u, up to 8 s worst case
```

The wall-clock period is $5 + T_a + T_u$, but the microphone only delivers data during the `record` call. Everything the engine emits during $T_a + T_u$ is *lost* — the system is deaf for that interval. Worse, the loss is not a fixed fraction: T_u is dominated by waiting for the ESP32's acknowledgement, with `ACK_TIMEOUT` = 4.0 seconds and `MAX_RETRIES` = 2, so a single unresponsive ESP32 can blind the recorder for eight seconds — more than an entire chunk. A communications fault would silently corrupt the *measurement* record. That coupling is unacceptable.

The remedy is the *producer-consumer* pattern with a bounded buffer, whose formulation in terms of semaphores dates to Dijkstra [dijkstra1968]. The producer (recorder) and the consumer (analyser) run as independent threads communicating through a synchronised queue, so the producer's timing is decoupled from the consumer's.

Python's Global Interpreter Lock is often raised as an objection to threading, and here it is not one. Every blocking operation in this system — `sd.wait()`, `wav.write()`, `ser.read()`, `time.sleep()` — releases the GIL, and the numerical work inside NumPy and SciPy releases it too. The threads are I/O-bound or release-the-GIL-bound almost everywhere, so they genuinely overlap.

9.2 The six threads

`main()` starts six daemon threads:

```
thread_specs = [
    ("health monitor",  uart_health_check),
    ("status monitor",  status_monitor),
    ("sync sender",     sync_sender),
    ("audio recorder",  recorder),
    ("signal analyzer", analyzer),
    ("input handler",   input_handler),
]
for name, target in thread_specs:
    t = threading.Thread(target=target, daemon=True, name=name)
    t.start()
```

Thread	Function	Role	Period
audio recorder	<code>recorder</code>	Produce 5 s WAV chunks; enqueue	continuous
signal analyzer	<code>analyzer</code>	Consume, run DSP, send UART	queue-driven
health monitor	<code>uart_health_check</code>	Watch UART health, force reconnect	60 s

Thread	Function	Role	Period
status monitor	<code>status_monitor</code>	Log queue depth and success rate	300 s
sync sender	<code>sync_sender</code>	Emit time-sync packet	60 s
input handler	<code>input_handler</code>	Interactive keyboard commands	blocking

Daemon status means all six are terminated abruptly at interpreter exit. The main thread does not busy-wait; it blocks on `shutdown_event.wait()` and is released by `q`, EOF, or `KeyboardInterrupt`.

The separation of `input_handler` into its own thread is a small but important detail. `input()` blocks indefinitely, and placing it on the main thread would prevent that thread from ever observing the shutdown event or a signal.

9.3 The bounded queue and overflow policy

```
ANALYZE_QUEUE_SIZE = 10
analyze_queue = queue.Queue(maxsize=ANALYZE_QUEUE_SIZE)
```

Ten slots at five seconds each gives fifty seconds of buffering — enough to absorb a long UART stall without loss.

The overflow policy is explicit and is chosen correctly:

```
if analyze_queue.full():
    print(f"Analyze queue full ... Dropping oldest item.")
    try:
        analyze_queue.get_nowait()
    except queue.Empty:
        pass
```

When the buffer saturates, the *oldest* pending chunk is discarded. The recorder never blocks. This is the right choice for a monitoring system: an audio recorder that stalls loses data irrecoverably, whereas a stale 50-second-old RPM reading has no value anyway. The policy degrades gracefully from “complete history” to “most recent data”, which is the correct degradation for real-time telemetry.

Note that the drop is not atomic with respect to the subsequent `put`. Between the `get_nowait()` and the `put`, the analyser could consume another item; the result is at worst one extra free slot, which is harmless. But `queue.Queue` provides no “put, evicting oldest” primitive, so a `collections.deque(maxlen=10)` guarded by a `Condition` would express the intent more directly.

9.4 Locking discipline

```
uart_lock = threading.Lock()    # serialises access to the serial port
init_lock = threading.Lock()    # serialises reconnection attempts
```

The two-lock structure is deliberate and correct. `uart_lock` guards all traffic on the port; without it, the analyser’s data packet and the sync sender’s sync packet could interleave mid-frame and

produce a byte stream that decodes as neither. The separate `init_lock` ensures `init_uart()` is not entered concurrently — which matters because `init_uart()` closes and reopens the port, and two threads doing that simultaneously would race on the global `ser`.

The critical section in `send_uart_packet` is long: it holds `uart_lock` across up to two transmit-and-wait-for-ACK cycles, each up to 4 seconds. A sync packet arriving during that window blocks for up to 8 seconds. Given that sync packets are generated only every 60 seconds, this is tolerable, but it is the mechanism by which UART trouble can back-pressure into the rest of the system.

One residual hazard: the reconnection check at the top of `send_uart_packet` reads the global `ser` *outside* `uart_lock`:

```
if ser is None or not ser.is_open:
    if not init_uart():
        ...
```

Another thread could close `ser` between the test and the subsequent use. The `try/except` inside the retry loop catches the resulting exception, so the failure mode is a logged error and a retry rather than a crash — but the check-then-act race is real.

9.5 The UART framed binary protocol

All three packet types share one frame:

```
+-----+-----+-----+-----+-----+
| START_BYTE | LENGTH | PAYLOAD (LENGTH) | CRC8 | 0x55 |
| 1 byte    | 1 byte | N bytes          | 1 byte | 1 byte |
+-----+-----+-----+-----+-----+
```

Type	Start	Payload	Length	ACK
Data	0xAA	seq(u16) + ts(u64) + rpm(f32) + status(u8) + freq(f32)	19	required
Record alert	0xAC	ts(u64)	8	none
Time sync	0xAB	ts(u64)	8	none

Payload construction, all little-endian:

```
payload = bytearray()
payload.extend(struct.pack('<H', seq_num))           # 2 sequence
payload.extend(struct.pack('<Q', timestamp))         # 8 ms since epoch
payload.extend(struct.pack('<f', float(rpm)))         # 4 IEEE-754 single
payload.extend(struct.pack('<B', int(engine_status))) # 1
payload.extend(struct.pack('<f', float(peak_freq)))  # 4
```

giving byte offsets `seq@0`, `ts@2`, `rpm@10`, `status@14`, `freq@15` and a total of 19, which is exactly how `mock_esp32.handle_data_packet` unpacks it.

The framing choices are individually justified. A **start byte** allows a receiver that has lost synchronisation to hunt for a frame boundary. An **explicit length** makes the receiver’s read deterministic and lets the frame carry variable payloads without ambiguity. A **CRC** detects corruption. A **distinct end byte** provides a second, independent structural check — if the byte at the expected terminal position is not 0x55, the frame is rejected even if the CRC happened to pass. And a **sequence number** allows loss and duplication to be detected, and lets the ACK identify *which* packet it acknowledges.

Little-endian ordering matches both the ARM Raspberry Pi and the Xtensa ESP32 natively, so neither end byte-swaps. The timestamp is packed with <Q, an *unsigned* 64-bit integer, so at millisecond resolution it will not overflow for approximately 585 million years.

9.6 CRC-8 over GF(2)

The checksum implementation is:

```
def calculate_crc8(data):
    crc = 0
    for byte in data:
        crc ^= byte
        for _ in range(8):
            if crc & 0x80:
                crc = (crc << 1) ^ 0x07
            else:
                crc = crc << 1
        crc &= 0xFF
    return crc
```

This is a bitwise, MSB-first CRC with initial value 0x00, no input or output reflection, and no final XOR — the parameterisation catalogued as **CRC-8/SMBUS**.

9.6.1 The algebra

The theory of cyclic codes for error detection was established by Peterson and Brown [peterson1961], and the exposition below follows their formulation.

A cyclic redundancy check is polynomial arithmetic over the finite field $\text{GF}(2) = \{0, 1\}$ with addition and subtraction both equal to XOR. A message of m bits $d_{m-1}, \dots, d_0$ is identified with the polynomial

$$D(x) = d_{m-1}x^{m-1} + \dots + d_1x + d_0 \in \text{GF}(2)[x].$$

The *generator polynomial* here is degree 8:

$$G(x) = x^8 + x^2 + x + 1,$$

whose coefficients are 1_0000_0111. The leading x^8 is implicit in the byte-sized register, which is why the code holds only the low eight bits 0x07.

The CRC is the remainder of the message, shifted left by the generator's degree, under division by G :

$$R(x) = (x^8 D(x)) \bmod G(x)$$

Transmitting $T(x) = x^8 D(x) + R(x)$ makes the codeword exactly divisible by $G(x)$, since in GF(2) addition and subtraction coincide. The receiver recomputes and compares — which is what `mock_esp32.read_packet` does:

```
expected_crc = calculate_crc8(payload)
if crc_byte[0] != expected_crc:
    log.warning(f"CRC mismatch: expected 0x{expected_crc:02X}, got 0x{crc_byte[0]:02X}")
    return None
```

Received data $T'(x) = T(x) + E(x)$ for an error polynomial $E(x)$. Since $G \mid T$, the check fails if and only if $G \nmid E$. **Undetected errors are exactly the nonzero multiples of $G(x)$** — so the code's strength is entirely a property of the generator's factorisation.

9.6.2 The bitwise loop as polynomial long division

Each iteration of the inner loop performs one step of long division. The register holds the current remainder. Shifting left multiplies by x ; if the bit shifted out of position 7 is 1, the running remainder has reached degree 8 and one copy of $G(x)$ must be subtracted, which over GF(2) is XOR with `0x07` (the x^8 term cancelling against the bit already shifted out). The outer `crc ^= byte` folds the next message byte into the register.

9.6.3 Detection guarantees

For $G(x) = x^8 + x^2 + x + 1$:

- **All single-bit errors.** $E(x) = x^i$, and G has more than one term, so $G \nmid x^i$.
- **All burst errors of length ≤ 8 bits.** A burst confined to b consecutive bits is $E(x) = x^i B(x)$ with $\deg B < b \leq 8 = \deg G$. Since $G(0) = 1$, $x \nmid G$, so $G \mid E$ would require $G \mid B$, impossible for a nonzero B of lower degree.
- **All odd-numbered bit errors.** $G(1) = 1 + 1 + 1 + 1 = 0$ over GF(2), so $(x + 1) \mid G$. Any E with an odd number of terms has $E(1) = 1 \neq 0$, hence is not divisible by $(x + 1)$ and therefore not by G .
- **Random errors.** Of 2^m possible error patterns, those divisible by G number about 2^{m-8} , so the residual undetected fraction is $2^{-8} = 0.39\%$.

The last figure is the honest limitation of an 8-bit CRC. Koopman and Chakravarty [koopman2004] analyse polynomial selection for exactly this class of embedded network and note that `0x07` — while standard and widely implemented — is not the Hamming-distance-optimal 8-bit polynomial for all payload lengths. For a 19-byte payload on a short, low-EMI board-to-board link at 115,200 baud, a 2^{-8} residual is entirely adequate, and interoperability with existing CRC-8/SMBUS implementations on the ESP32 side is worth more than a marginal improvement in Hamming distance.

Note that the CRC covers the *payload only*, not the start byte or length byte. A corrupted length byte would cause a framing error rather than a CRC failure — caught by the end-byte check, but less directly.

9.7 The ACK handshake

Only Data packets are acknowledged. The ESP32 replies with three bytes:

```
+-----+-----+-----+
| 0x06 | seq_low | seq_high |
+-----+-----+-----+
```

0x06 is ASCII ACK. The sender validates both the marker and the sequence:

```
ser.timeout = ACK_TIMEOUT
response = ser.read(3)
if len(response) == 3 and response[0] == 0x06:
    ack_seq = struct.unpack('<H', response[1:3])[0]
    if ack_seq == seq_num:
        ... # success
```

The sequence check is what makes the handshake meaningful. Without it, a stale ACK for an earlier packet would be accepted as confirmation of the current one. With `MAX_RETRIES = 2` and `ACK_TIMEOUT = 4.0`, a persistently unresponsive ESP32 costs at most about 8 seconds per chunk before the packet is abandoned and a failure recorded.

The sequence counter wraps at 16 bits:

```
packet_seq_num = (packet_seq_num + 1) % 65536
```

At one packet per five seconds, that is a wrap every 91 hours — long enough to be unambiguous within any realistic flight or ground-run session.

9.8 Health monitoring and recovery

The `UARTHealthMonitor` class tracks `consecutive_failures`, `total_attempts`, `total_successes`, and `last_successful_send`, and exposes a reconnection policy:

```
def should_reconnect(self):
    time_since_success = time.time() - self.last_successful_send
    return (self.consecutive_failures >= self.max_consecutive_failures or
            time_since_success > CONNECTION_TIMEOUT)
```

with `MAX_CONSECUTIVE_FAILURES = 5` and `CONNECTION_TIMEOUT = 300` seconds. This is a two-clause policy, and both clauses are needed. The failure count catches a fast, loud failure such as an unplugged cable. The elapsed-time clause catches a *silent* failure — a port that accepts writes without error but is not actually connected to anything — which would otherwise never increment the failure counter, because `ser.write()` to a disconnected but open port succeeds.

The port is opened with `exclusive=True`, preventing a second process from attaching to `/dev/serial0` and stealing bytes — a failure that is otherwise extremely difficult to diagnose from the application's point of view.

10 Code Implementation

10.1 Configuration constants

All tunable parameters are module-level constants at the top of `rpi_test_final.py`.

Constant	Value	Units	Governs
RECORD_SECONDS	5	s	Chunk duration; sets $\Delta f = 0.2$ Hz and latency
SAMPLE_RATE	44100	Hz	Acquisition rate
DECIMATED_RATE	500	Hz	Target rate; gives $M = 88$
EVENTS_PER_CYCLE	4	events/rev	Assumed harmonic order
AUDIO_DIR	./recordings	—	WAV storage
UART_PORT	/dev/serial0	—	Serial device
UART_BAUDRATE	115200	baud	Line rate
ANALYZE_QUEUE_SIZE	10	chunks	50 s of buffering
ACK_TIMEOUT	4.0	s	ACK wait
MAX_RETRIES	2	—	Transmit attempts
INTER_PACKET_DELAY	0.05	s	Retry backoff base
HEALTH_CHECK_INTERVAL	60	s	Health poll period
MAX_CONSECUTIVE_FAILURES	3	—	Reconnect trigger
CONNECTION_TIMEOUT	300	s	Silent-failure trigger
SYNC_INTERVAL	60	s	Time-sync period
CLEAN_INTERVAL_DAYS	7	days	Cleanup period
MIN_FREE_SPACE_GB	1.0	GB	Cleanup trigger
DISK_SPACE_THRESHOLD	0.1	fraction	Cleanup trigger
START_BYTE_DATA	0xAA	—	Data frame marker
START_BYTE_SYNC	0xAB	—	Sync frame marker
START_BYTE_RECORD	0xAC	—	Record-alert marker
END_BYTE	0x55	—	Frame terminator

0x55 is 0b01010101, a maximally alternating pattern that is unlikely to arise from a stuck line or a framing error, which is why it is a conventional choice for a terminator.

10.2 The Datum container

The pipeline passes a single mutable object between stages:

```
class Datum:
    def __init__(self, audio_array, sample_rate):
        self._raw_data = {RawDatumKey.AUDIO_ARRAY: audio_array,
                          RawDatumKey.SAMPLE_RATE: sample_rate}
        self._derived_data = {}
```

with string-keyed accessors `get_raw_datum`, `set_raw_datum`, `get_derived_data`, `set_derived_data`. Raw and derived data are kept in separate dictionaries, which enforces the invariant that a pro-

cessing stage never mutates its input — it only adds derived products. That in turn is what makes stages composable in any order and makes the whole record available for logging at the end.

The library version in `audio_src/DataIO/Representation/Datum.py` is richer: it uses `Enum` keys rather than string constants (giving type safety and IDE autocompletion), carries a `DatumType` discriminator, and supports labels and label names for supervised learning. The deployed script's string-keyed variant is a deliberate simplification to avoid the library import.

10.3 The pipeline

```
class FeatureEngineeringPipeline:
    def __init__(self):
        self.__feature_engineering_blocks = []
        self.__output_blocks = []

    def add_block(self, block): self.__feature_engineering_blocks.append(block)
    def add_output_block(self, block): self.__output_blocks.append(block)

    def run(self, datum):
        for block in self.__feature_engineering_blocks:
            datum = block.extract_features(datum)
        for block in self.__output_blocks:
            datum = block.extract_features(datum)
        return datum
```

A two-phase ordered composition of `FeatureExtraction` objects. The distinction between feature blocks and output blocks is one of intent: feature blocks derive signal representations, output blocks derive reportable physical quantities.

Assembly in `analyzer()`:

```
pipeline = FeatureEngineeringPipeline()
pipeline.add_block(Decimation(target_rate=DECIMATED_RATE))
pipeline.add_block(FrequencyPeakFinder())
pipeline.add_output_block(RPM(events_per_crankshaft_cycle=EVENTS_PER_CYCLE))
```

so the DSP call chain is exactly

$$\text{decimate} \rightarrow \text{window} + \text{FFT} + \text{argmax} \rightarrow \text{RPM} + \text{status}.$$

Note that `RPM.__init__` defaults to `events_per_crankshaft_cycle=2` — the kinematically correct value — and is overridden to 4 at every call site. A reader of the class alone would draw the wrong conclusion about the deployed behaviour.

10.4 The recorder thread

```
audio = sd.rec(int(RECORD_SECONDS * SAMPLE_RATE),
               samplerate=SAMPLE_RATE, channels=1,
               dtype='int16', device=MIC_DEVICE)
```

```
sd.wait()
wav.write(filepath, SAMPLE_RATE, audio.flatten())
analyze_queue.put((filepath, record_start_time, packet_seq_num))
```

`sd.rec` allocates the buffer and starts a non-blocking capture; `sd.wait()` blocks until it completes, releasing the GIL throughout. Recording is 16-bit mono, which gives a theoretical dynamic range of $6.02 \times 16 + 1.76 = 98$ dB — far more than any microphone’s own noise floor requires.

The enqueued item is a *file path*, not an array. This is a good decision: the WAV on disk is the authoritative record, the queue holds only tuples of a few tens of bytes rather than 441 kB arrays, and the recording survives an analyser crash. The cost is a disk round-trip per chunk, which is trivially absorbed by the page cache.

Before each recording a Record-alert packet is emitted:

```
record_start_time = get_timestamp()
send_record_alert(record_start_time)
```

so the ESP32 learns the *start* time of a chunk immediately, roughly five seconds before the corresponding Data packet arrives. That lets the ESP32 distinguish “the Pi is recording but analysis is slow” from “the Pi has stopped” — a genuinely useful liveness signal.

Disk hygiene is handled in the same loop: `check_disk_space()` triggers `cleanup_old_files()`, which sorts by modification time and deletes oldest-first until the threshold clears.

10.5 The analyzer thread

```
filepath, record_timestamp, seq_num = analyze_queue.get()
sample_rate, data = wav.read(filepath)
if len(data.shape) > 1:
    data = data.flatten()
datum = Datum(audio_array=data, sample_rate=sample_rate)
datum = pipeline.run(datum)
success = send_uart_packet(record_timestamp, datum, seq_num)
```

`analyze_queue.get()` blocks indefinitely when empty, so the thread consumes no CPU while idle. The sample rate is read *from the file* rather than assumed from the constant, which is why the pipeline transparently handles the 48,000 Hz simulation audio as well as the 44,100 Hz live path.

The timestamp forwarded to the ESP32 is `record_timestamp` — the moment recording *began* — not the moment analysis finished. This is correct: it is the epoch to which the measurement refers, and it makes the telemetry timeline reflect physical time rather than processing latency.

The transmitted RPM is the *chunk mean* in the sense that the FFT integrates over the whole five seconds. It is not an instantaneous value, and a consumer plotting it should treat each sample as covering the interval `[record_timestamp, record_timestamp + 5s]`.

10.6 Evolution across iterations

The repository preserves three earlier versions, and the trajectory is informative.

rpi_test.py (500 lines). The minimal working system. `EVENTS_PER_CYCLE = 2`, the kinematic value. Hann window, dual raw/decimated analysis, all three packet types, three threads.

rpi_test_fft.py (903 lines). Adds diagnostic instrumentation: full FFT export to JSON, zero-padding to the next power of two, `scipy.signal.find_peaks` for multi-peak detection, an `export_fft_data` function, and `should_export_fft` triggering logic. Still `EVENTS_PER_CYCLE = 2`. This is the *investigative* version — the one that made the order structure visible and that presumably motivated the change to 4.

rpi_test_new.py (1600 lines). The most ambitious. Introduces `AircraftEngineType`, `AircraftEngineState` (with `GROUND_IDLE`, `TAXI_POWER`, `RUNUP`, `TAKEOFF_POWER`, `CRUISE_POWER`), `AlertLevel`, and a configuration class hierarchy:

```
class PistonEngineConfig(AircraftEngineConfig):
    def __init__(self, model="LYCOMING_I0360"):
        self.events_per_cycle = 2          # 4-stroke engine
        self.freq_range_min = 15
        self.freq_range_max = 200
        ...
        self.state_thresholds = {
            AircraftEngineState.STARTING:      (200, 600),
            AircraftEngineState.GROUND_IDLE:    (self.idle_min, self.idle_max),
            AircraftEngineState.TAXI_POWER:     (900, 1200),
            AircraftEngineState.RUNUP:          (1400, 1800),
            AircraftEngineState.TAKEOFF_POWER:  (2300, self.redline),
            AircraftEngineState.CRUISE_POWER:   (self.cruise_min, self.cruise_max),
        }
```

with a `TurbopropEngineConfig` using `events_per_cycle = 1` for continuous combustion and percentage-RPM reporting. It also raises `DECIMATED_RATE` to 1000 Hz (“Higher for aircraft”) and adds a `HARMONIC_CONTENT` derived quantity and hysteresis on state transitions.

rpi_test_final.py (791 lines). A deliberate retreat from `rpi_test_new.py`’s scope back to `rpi_test.py`’s, keeping only the operationally proven parts: the health monitor, the reconnection logic, and the six-thread structure. The single substantive DSP change from `rpi_test.py` is `EVENTS_PER_CYCLE 2` to 4.

The trajectory simple to instrumented to ambitious to simple-but-hardened is a healthy one. But it is worth recording that `rpi_test_new.py` already contained, in its per-engine configuration hierarchy, the mechanism needed to fix the generalisation problem of Section 13 — and that mechanism was discarded along with the rest of the scope. Section 14 proposes recovering it.

10.7 The extended library

10.7.1 Module structure

```
audio_src/
  DataIO/
    Communication/  AudioStream.py, RTAudioStreamSimulator.py, Interface.py
    Representation/ Datum.py, data_point.py
  FeatureEngineering/
```

```

Filters/          Butterworth.py, Decimate.py, Other.py, Interface.py
Spectrograms/    Fourier.py, Interface.py
FeatureExtraction/ EngineAnalysis.py, Interface.py
FeatureEngineeringPipeline.py
Sensors/          Vibration_Sensor.py, modified_vibration.py

```

Every subsystem is defined by an abstract base class in its `Interface.py`: `CommunicationInterface` (`read_sample/write_sample`), `Filter` (`apply_filter`), `FrequencySpectrumCalculator` (`make_spectrum/make_timeseries`), and `FeatureExtraction` (`extract_features`). This is a clean design — a new filter or spectrum estimator is a drop-in — and it is what makes the alternatives in Section 14 cheap to try.

10.7.2 Decimate.py

Two implementations. `AntiAliasingDecimation` wraps `scipy.signal.decimate(signal, q, n=order)` with an explicit order-8 default, matching what the deployed script gets implicitly. `Resample` instead uses `scipy.signal.resample`, which is FFT-based:

```

resampled_length = int(len(signal) * self.__new_rate / sample_rate)
return scipy.signal.resample(signal, resampled_length)

```

This performs ideal bandlimited interpolation — it takes the FFT, truncates or zero-pads the spectrum, and inverts — so it supports arbitrary (non-integer) rate ratios exactly and introduces no filter phase distortion at all. Its cost is $\mathcal{O}(N \log N)$ rather than the IIR path’s $\mathcal{O}(N)$, and it assumes circular periodicity, which produces edge artefacts on non-periodic records. `EngineAnalysis.Decimation` uses `Resample`, so the *library* pipeline resamples to an exact 500 Hz while the *deployed* pipeline decimates by 88 to a nominal 501 Hz. This is another divergence between the two paths that should be reconciled.

10.7.3 Fourier.py

Three classes. `FFT` wraps `scipy.fft.fft` with cached frequency bins. `RFFT` uses the real-input transform and adds normalisation:

```

time_series_normalized = time_series / np.max(np.abs(time_series))
yf = scipy.fft.rfft(time_series_normalized, n=self.__nfft)[1:]
self.__freq = scipy.fft.rfftfreq(len(time_series), 1 / self.__fs)[1:]
yf_norm = yf / np.max(np.abs(yf))
return yf_norm

```

Two things happen here that matter for the classifier that consumes it. The `[1:]` slice discards the DC bin, removing any constant offset. And the input and output are each normalised to unit peak, which makes the spectrum *amplitude-invariant* — a necessary property for a classifier that must work at any microphone gain and any distance, since only spectral *shape* should carry the decision. It also means the representation carries no absolute level information at all, so a “quiet but present” engine and a “loud” one are indistinguishable to it. Note the normalisation will divide by zero on a digitally silent input.

`STFT` wraps `scipy.signal.stft` with a Hann window and configurable `nperseg` and `noverlap`, plus `make_timeseries` via `istft`.

10.7.4 EngineAnalysis.EngineState: the TFLite classifier

The most sophisticated component in the repository. It classifies engine state from the RFFT spectrum using a pre-trained TensorFlow Lite model, with temporal voting.

```
self.interpreter = tf.lite.Interpreter(model_path=model_path)
self.interpreter.allocate_tensors()
self.input_details = self.interpreter.get_input_details()
self.output_details = self.interpreter.get_output_details()
self.rfft_calculator = RFFT()
self.__buffer = np.array([])
self.__buffer_seconds = 1
self.__buffer_results = []
self.__buffer_results_dimension = 70
```

Per-window inference:

```
self.__buffer = np.concatenate((self.__buffer, signal_array), axis=None)
self.__buffer = self.__buffer[-int(sample_rate * self.__buffer_seconds):]

if len(self.__buffer) == int(sample_rate * self.__buffer_seconds):
    rfft_repr = self.rfft_calculator.make_spectrum(self.__buffer)
    amplitude_spectrum = np.array([np.abs(rfft_repr)], dtype=np.float32)
    self.interpreter.set_tensor(self.input_details[0]['index'], amplitude_spectrum)
    self.interpreter.invoke()
    pred_vector = self.interpreter.get_tensor(self.output_details[0]['index'])
    pred = np.argmax(pred_vector)
else:
    pred = 1
```

then the vote:

```
self.__buffer_results.append(pred)
self.__buffer_results = self.__buffer_results[-int(self.__buffer_results_dimension):]
pred = np.argmax(np.bincount(self.__buffer_results))
signal.add_derived_data(DerivedDataKey.ENGINE_STATE, self.label_literals[pred])
```

Two points of precision before analysing this. `np.argmax(np.bincount(...))` returns the *mode* of the buffer — a plurality vote, not strictly a majority vote, which are the same thing only for two classes; with three labels (on / off / fault, per the README) a class can win with well under half the votes. And the buffer holds 70 *invocations*, not 70 seconds: its temporal span is 70 times the duration of whatever chunk the caller supplies. It is 70 seconds only if chunks are one second long, as they are for the default `AudioStream` and `RTAudioStreamSimulator` buffer size; under the deployed script’s five-second chunking it would span 350 seconds.

Why temporal voting. The classifier sees one second of audio. In one second a great deal can go wrong: a radio transmission, a gust, a cough, a moment when the engine is masked by propeller noise. An instantaneous classification is therefore noisy, and a raw per-window output would produce a state indicator that flickers between “on” and “fault” many times a minute — which is worse than useless, because it destroys operator trust in the indicator entirely.

The rolling vote over $K = 70$ windows converts this noisy instantaneous classifier into a stable one. Taking the two-class case, where plurality and majority coincide: if the per-window classification is correct with probability p and errors are independent, the vote is correct with probability

$$P_{\text{correct}} = \sum_{i=\lceil (K+1)/2 \rceil}^K \binom{K}{i} p^i (1-p)^{K-i},$$

which for $K = 70$ and even a mediocre $p = 0.7$ evaluates to 0.99964. The vote turns a 30%-error classifier into a 0.036%-error one — an improvement of nearly three orders of magnitude. At $p = 0.6$ it still yields 0.943.

The cost is *latency*. With one-second windows the buffer spans 70 seconds, so a genuine state change — an engine shutdown — takes about 35 seconds to command a majority and up to 70 to become unanimous. For the intended purpose (logging whether the engine is running) this is entirely acceptable; for anything requiring prompt fault annunciation it would not be. The independence assumption is also optimistic: a 70-second masking event correlates errors across the whole buffer, so the true improvement is smaller than the binomial figure. The design is nonetheless a textbook-correct application of temporal smoothing to a noisy detector, and the buffer length is a directly tunable latency-versus-stability knob.

Before the buffer fills, `pred = 1` is used as a neutral placeholder, so the vote is biased toward class 1 during start-up. Naming class 1 “unknown” or excluding placeholder votes would be cleaner.

10.7.5 RTAudioStreamSimulator

```
for i in range(self.__buffer_dim, len(self.__audio_array), self.__buffer_dim):
    a = self.__audio_array[i - self.__buffer_dim:i]
    yield a
```

Replays a recorded file as if it were a live stream, chunk by chunk, behind the same `CommunicationInterface` that the live `AudioStream` implements. Because it is interface-compatible, the entire downstream pipeline — including stateful components such as `TunableFilter` and `EngineState` — can be exercised deterministically and repeatably against recorded audio. For a system whose stateful components cannot otherwise be tested without an aircraft, this is the single most valuable test-infrastructure component in the repository, and it is underused.

10.7.6 Sensors/

`Vibration_Sensor.py` reads an I2C accelerometer at address 0x53 and thresholds $|a_x| + |a_y| + |a_z|$ into three bands. `modified_vibration.py` targets the ADXL345 specifically, with `POWER_CTL/DATA_FORMAT` initialisation, a 100-sample calibration routine to remove the gravity offset, conversion at 4 mg/LSB, and a 0.2 g excess-vibration warning. Neither is integrated into the audio pipeline. Section 14 argues they should be, because an accelerometer bolted to the engine mount observes the same order structure with a far better signal-to-interference ratio than a cabin microphone, and would resolve the order ambiguity directly.

11 Experiments and Validation

11.1 Why a staged strategy

An embedded system of this kind has three qualitatively different dependency classes, and a test that spans all three cannot localise a failure. If a full end-to-end run produces a wrong RPM, the cause could be the DSP, the microphone, the serial wiring, the ESP32 firmware, or the packet encoding — and the test result alone does not say which.

The repository therefore implements a three-level ladder, each level adding exactly one dependency class:

Level	Script	Adds	Runs on
1	<code>test_pipeline.py</code>	Nothing (pure computation)	Any machine
2	<code>mock_esp32.py</code> + <code>socat</code>	Serial transport, framing, CRC, ACK	Any machine
3	<code>test_hardware.py</code>	Microphone, real UART, real ESP32	Raspberry Pi

The value of the ladder is *bisection*. A Level-1 failure is a DSP bug. A Level-1 pass with Level-2 failure is a protocol bug. Levels 1 and 2 passing with Level 3 failing is a hardware or wiring problem. Each level is also progressively less convenient to run — Level 1 needs nothing, Level 3 needs an aircraft-adjacent bench — so the ladder also orders tests by how often they can realistically be executed.

11.2 Level 1: DSP pipeline

`test_pipeline.py` imports the pipeline classes directly from `rpi_test_final.py` — the same code that is deployed, not a copy:

```
from rpi_test_final import (
    Datum, FeatureEngineeringPipeline, Decimation,
    FrequencyPeakFinder, RPM, DerivedDataKey,
    DECIMATED_RATE, EVENTS_PER_CYCLE,
)
```

and rebuilds the identical pipeline. Four cases are exercised.

Real recordings.

```
TEST_CASES = [
    ("1000_rpm.wav", 1, 800, 1200), # 2nd harmonic at 66.96 Hz -> ~1004 RPM
    ("vol01.wav", 1, 0, 5000),
]
```

Checks are that RPM and frequency are finite, that status is in $\{0,1\}$, that status matches expectation, and that RPM falls in the stated range.

Silence. `np.zeros(44100 * 5)` must give exactly 0.0 RPM and status 0. This tests the zero-guard path in the peak finder and the threshold in the RPM block.

Synthetic tone. A pure 83.3 Hz sinusoid, chosen so that

$$\Omega = \frac{83.3 \times 60}{4} = 1249.5 \text{ rpm},$$

with an accepted range of [1000, 1500]. This is the *only* case with an analytically known ground truth, and it is the one that validates the arithmetic of the RPM formula end-to-end independently of any engine.

11.3 Level 2: virtual ESP32

Level 2 uses `socat` to create a pair of connected pseudo-terminals:

```
socat -d -d pty,raw,echo=0 pty,raw,echo=0
```

`mock_esp32.py` attaches to one end and `rpi_test_final.py` to the other, with `UART_PORT` temporarily repointed. The mock reimplements the receiver independently:

```
start = port.read(1)
if start_byte not in (START_BYTE_DATA, START_BYTE_RECORD, START_BYTE_SYNC): ...
length = port.read(1)[0]
payload = port.read(length)
crc_byte = port.read(1); end_byte = port.read(1)
if end_byte[0] != END_BYTE: ...
if crc_byte[0] != calculate_crc8(payload): ...
```

then decodes and acknowledges:

```
seq      = struct.unpack_from('<H', payload, 0)[0]
timestamp = struct.unpack_from('<Q', payload, 2)[0]
rpm      = struct.unpack_from('<f', payload, 10)[0]
status   = payload[14]
freq     = struct.unpack_from('<f', payload, 15)[0] if len(payload) >= 19 else 0.0
ack = bytes([0x06]) + struct.pack('<H', seq)
port.write(ack)
```

What this validates that Level 1 cannot: byte-level framing over a real character device; that the CRC computed by the sender is reproduced by an *independent* implementation of the same algorithm; that the struct offsets agree between the two ends; that the ACK format and sequence echo are correct; and that the sender's retry and timeout logic behaves when the ACK is delayed or absent.

The independence of the two CRC implementations is the point. Had the sender's CRC been wrong — say, the wrong polynomial — a receiver *copied* from the sender would agree with it perfectly and the test would pass. The mock's version is written separately in `mock_esp32.calculate_crc8` and is therefore a genuine cross-check of the algorithm, not merely of the transport.

It also serves as executable reference documentation for whoever writes the real ESP32 firmware.

11.4 Level 3: on-device hardware

`test_hardware.py` runs on the Pi and checks the three hardware dependencies in order.

Microphone. Enumerates input devices, records two seconds, and checks that data came back and is not silent:

```
rms = float(np.sqrt(np.mean(audio.astype(np.float32) ** 2)))
check("recording returned data", audio.size > 0, f"{audio.size} samples")
check("audio is not silent", rms > 10, f"RMS={rms:.1f}")
```

The RMS check is what distinguishes a working microphone from a device that enumerates, opens, and returns zeros — a common and otherwise silent failure with misconfigured I2S.

Pipeline on live audio. The recorded clip is pushed through the same pipeline and checked for finite outputs. Note this is only a *sanity* check — the ambient office noise it captures has no known RPM — so it verifies the code path executes on real hardware, not that the answer is right.

UART. Checks `/dev/serial0` exists, opens it exclusively, writes a sync packet and confirms the byte count, then writes a Data packet and waits for an ACK:

```
seq = 0xBEEF & 0xFFFF
payload = struct.pack('<H', seq) + struct.pack('<Q', ts)
payload += struct.pack('<f', 1234.5) + struct.pack('<B', 1) + struct.pack('<f', 41.15)
packet = bytearray([0xAA, len(payload)]) + payload + bytearray([_crc8(payload), 0x55])
ser.write(packet); ser.flush()
ser.timeout = 3.0
response = ser.read(3)
got_ack = (len(response) == 3 and response[0] == 0x06 and
           struct.unpack('<H', response[1:3])[0] == seq)
```

The distinctive sequence value `0xBEEF` makes it immediately obvious in a logic analyser trace or an ESP32 log which packet is the test packet.

The script exits 0 only if every check passed, making it usable as a pre-flight gate in a deployment script.

11.5 What the ladder does not cover

Three gaps should be recorded honestly.

No ground-truth RPM. No level compares the estimate against a reference tachometer. Level 1's `1000_rpm.wav` has only a filename as its label, and `vo1o1.wav` has no label at all — which is why its accepted range is the vacuous `[0, 5000]`. The synthetic tone is the only analytic ground truth, and it tests the formula rather than the engine model.

Level 1 does not test the deployed configuration. `test_pipeline.py` processes each WAV file *in its entirety* — 73.6 seconds for `1000_rpm.wav`. The deployed system processes *five-second* chunks. These are materially different operating points, and Section 12 shows that the deployed configuration fails on cases where the whole-file configuration passes. The test therefore validates an operating point the system never runs at.

Loose tolerances. The `vo1o1.wav` band of `[0, 5000]` rpm cannot fail. A test that cannot fail provides no information.

12 Results and Interpretation

All results in this section were produced by executing the repository’s own pipeline classes on the repository’s own audio, under NumPy 2.0.0 and SciPy 1.13.1.

12.1 Documented Level-1 results, verified

The README states four expected outcomes. Each was re-derived by running the pipeline, and each is confirmed.

Input	Peak (decimated)	RPM reported	Status	README
1000_rpm.wav	66.971 Hz	1004.56	ON (1)	~1004, ON
volol1.wav	84.237 Hz	1263.56	ON (1)	~1263, ON
silent (zeros)	0.000 Hz	0.00	OFF (0)	0, OFF
83.3 Hz tone	83.367 Hz	1250.50	ON (1)	~1250, ON

Arithmetic self-consistency with $\Omega = 15\hat{f}$:

- $15 \times 66.9708 = 1004.56$. Matches.
- $15 \times 84.2373 = 1263.56$. Matches.
- $15 \times 0 = 0$, and $0 > 100$ is false, so status 0. Matches.
- $15 \times 83.3667 = 1250.50$. Matches; and the ideal $15 \times 83.3 = 1249.5$ differs only by the 0.1997 Hz bin quantisation, exactly as Section 4 predicts.

All four documented results are reproducible and internally consistent with the RPM formula at EVENTS_PER_CYCLE = 4.

Signal parameters for the record:

File	f_s	Duration	M	$f_s^{(d)}$	N_d	Δf
1000_rpm.wav	48,000 Hz	73.643 s	96	500 Hz	36,822	0.0136 Hz
volol1.wav	48,000 Hz	14.933 s	96	500 Hz	7,467	0.0670 Hz
synthetic	44,100 Hz	5.000 s	88	501 Hz	2,506	0.1999 Hz

Note that both real recordings are 48 kHz, for which $M = 96$ divides exactly and the 0.027% rate-truncation bias of Section 5 does not arise.

12.2 The order spectrum of volol1.wav

Section 7 established the order structure of 1000_rpm.wav. Applying the same analysis to the in-flight recording volol1.wav, with half-order base $f_{0.5} = 21.059$ Hz:

Order o	Frequency (Hz)	Relative amplitude
0.5	21.059	0.042
1.0	42.118	0.033
1.5	63.177	0.314
2.0	84.236	1.000

Order o	Frequency (Hz)	Relative amplitude
2.5	105.295	0.110
3.0	126.354	0.176
3.5	147.413	0.128
4.0	168.472	0.907
4.5	189.531	0.078
5.0	210.590	0.049

The structure is the same family type as `1000_rpm.wav`: a half-order series with dominant even orders 2 and 4, and moderate odd half-orders indicating cylinder imbalance. But **the ranking of orders 2 and 4 is reversed**. In `1000_rpm.wav` order 4 leads order 2 by 0.23 dB; here order 2 leads order 4 by

$$20 \log_{10}(1.000/0.907) = 0.85 \text{ dB}.$$

The consequence is arithmetic. The peak-picker selects 84.236 Hz, which is order 2, and the formula applies $\nu = 4$:

$$\hat{\Omega} = \frac{60 \times 84.236}{4} = 1263.5 \text{ rpm},$$

whereas the order-2 assignment gives

$$\Omega = \frac{60 \times 84.236}{2} = 2527.1 \text{ rpm},$$

which is also what the half-order base gives directly: $120 \times 21.059 = 2527.1 \text{ rpm}$.

The reported 1263.6 rpm for `vol01.wav` is therefore an exact factor-of-two underestimate, and the true crankshaft speed is approximately 2527 rpm.

The evidence for this reading is threefold and, in the author's judgement, strong, though it should be stated that no tachometer reference exists for this file.

First, *internal consistency*. The observed family spacing of 21.059 Hz is the order-0.5 component. If instead 84.236 Hz were order 4 (making 21.059 Hz order 1), the spectrum would contain no half-order components at all — which would be anomalous for a four-stroke and inconsistent with `1000_rpm.wav`, where the same family type appears with the same spacing relationship.

Second, *the shape of the family*. Under the order-2 reading, the strong components are at orders 2 and 4 (the firing frequency and its second harmonic) with weaker odd half-orders — precisely the `1000_rpm.wav` pattern. Under the order-4 reading, the strong components would be at orders 4 and 8 with nothing at 2, which would require the firing fundamental to be absent while its harmonics are dominant.

Third, *plausibility*. 2527 rpm is a physically sensible Rotax 912 speed. 1263 rpm is below the engine's idle and is not a speed at which an aircraft flies.

It also explains the loose test tolerance. `test_pipeline.py` accepts $[0, 5000]$ rpm for this file — a range wide enough to pass whichever assignment is correct. The tolerance is not carelessness so much as a tacit admission that the correct answer was not known.

12.3 Chunk-level behaviour: the deployed configuration

The results above process each file whole. The deployed system processes five-second chunks. Repeating the analysis chunk by chunk, exactly as deployed:

1000_rpm.wav, 14 non-overlapping 5-second chunks. Selected peak frequencies:

64.2 64.6 65.4 65.8 66.4 66.8 66.8 66.8
67.6 67.4 34.0 33.8 66.6 33.4 (Hz)

Eleven chunks select a frequency near 66 Hz (order 4). **Three chunks — numbers 11, 12, and 14 — select approximately 33.7 Hz, which is order 2.** Converting with $\nu = 4$:

Subset	n	Mean RPM	SD	Range
Order-4 chunks	11	993.3	15.7	963–1014
Order-2 chunks (flipped)	3	505	—	501–510
All chunks	14	888.9	200.4	501–1014

The order-flip rate is $3/14 = 21.4\%$, and each flip produces a 50% RPM error.

This is the central experimental result of this document. The recording on which `EVENTS_PER_CYCLE = 4` was calibrated, processed in the configuration the system actually deploys, produces a grossly wrong answer on more than one chunk in five.

The whole-file analysis conceals this completely: integrating over 73.6 seconds lets the marginally stronger order 4 accumulate enough energy to win overall, so `test_pipeline.py` reports a clean pass. The failure is visible only at the five-second granularity the system actually uses. This is exactly the validation gap identified at the end of Section 11.

The 15.7 rpm scatter among the eleven correctly-assigned chunks deserves separate comment. It is more than five times the 3.0 rpm bin quantisation of Section 4, so it is not a resolution artefact. Inspecting the sequence, the peak rises monotonically from 64.2 Hz through 67.6 Hz over the first nine chunks — this is a genuine engine speed increase of about 5%, resolved correctly. The estimator is tracking a real transient, not jittering.

vol01.wav, 2 chunks. Selected peaks: 84.2 Hz and 168.4 Hz — order 2 for the first chunk and order 4 for the second. Reported RPM: 1263 and 2526. The same recording, five seconds apart, produces answers differing by exactly a factor of two. The second chunk’s 2526 rpm happens to be the *correct* value, arrived at by the compensating coincidence of a wrong peak selection and a matching wrong order constant.

12.4 Multi-order estimation: a measured alternative

If the failure is that a single peak does not determine the order, the remedy is to use the *pattern* of peaks. A harmonic-template (order-comb) estimator scores each candidate speed Ω by summing spectral magnitude at all the orders that speed predicts:

$$S(\Omega) = \sum_{o \in \mathcal{O}} \max_{|f - o\Omega/60| \leq 2\Delta f} |X(f)|, \quad \hat{\Omega} = \arg \max_{\Omega} S(\Omega),$$

with the physically motivated order set

$$\mathcal{O} = \{0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 6\}$$

and a 1 rpm search grid over [400, 3500] rpm. This is a direct, cheap implementation of the multi-order fusion principle from the tachless order tracking literature [lu2019; peeters2019].

Run on the same five-second chunks:

1000_rpm.wav:

1932 966 975 984 992 1000 1000 998
1010 1006 1022 1015 995 998 (rpm)

Estimator	Failures	Mean (valid)	SD (valid)
Single-peak argmax, $\nu = 4$	3/14 (21.4%)	993.3	15.7
Harmonic template	1/14 (7.1%)	997.0	14.8

Every chunk that the single-peak estimator got wrong, the template estimator got right. Chunks 11, 12, and 14 — which the argmax reported as 501, 507, and 501 rpm — are estimated by the template at 1022, 1015, and 998 rpm. The template does make one different error, an octave error on chunk 1 (1932 rpm, close to twice the true speed), which is the characteristic failure mode of comb estimators when the even orders happen to align with the doubled fundamental; a subharmonic penalty term or a continuity constraint would suppress it.

vol01.wav: the template returns 2522 rpm on *both* chunks — standard deviation exactly zero, against the single-peak estimator’s 631 rpm standard deviation on the same two chunks. This is independent confirmation of the 2527 rpm reading derived from the order table above: two completely different methods, one based on the family spacing and one on comb-scoring, agree to within 0.2%.

The template estimator requires no tachometer, no training data, no state, and no model of the engine beyond its cylinder count and stroke cycle — which are known constants for any given installation. Its cost is one FFT (already computed) plus a few thousand array lookups per chunk, well within the Pi’s budget. Section 14 recommends it as the direct replacement for the current estimator.

12.5 Summary of findings

1. All four documented Level-1 results are reproducible and arithmetically consistent with $\Omega = 15\hat{f}$.
2. The order structure of both recordings is a textbook four-cylinder four-stroke half-order family, confirming the physical model of Section 7.
3. `EVENTS_PER_CYCLE = 4` is correct for `1000_rpm.wav` whole-file, but by a margin of only 0.23 dB.

4. `EVENTS_PER_CYCLE = 4` is *wrong by a factor of two* for `vol01.wav`, whose true speed is approximately 2527 rpm.
5. In the deployed five-second-chunk configuration, the order-flip rate on `1000_rpm.wav` is 21.4%, each flip causing a 50% error.
6. A harmonic-template estimator eliminates every one of those failures and stabilises `vol01.wav` completely, at negligible additional cost.

13 Limitations and Failure Cases

13.1 The fixed order assumption

The system’s dominant limitation, established quantitatively in Section 12, is that `EVENTS_PER_CYCLE` is a compile-time constant while the order it refers to is a run-time property of the signal.

The failure is *silent*. There is no confidence measure, no runner-up magnitude, no plausibility envelope. A 50% error is emitted with exactly the same apparent authority as a correct reading, and the downstream ESP32 has no basis on which to distinguish them.

The failure is also *not rare*. 21.4% of chunks on the calibration recording, and 50% of chunks (1 of 2) on the in-flight recording.

And the failure is *bounded but large*: because the competing candidates are harmonically related, the error is always a rational multiple — typically exactly $\times 2$ or $\times \frac{1}{2}$. This is at least a structured error, which is what makes it detectable by a continuity check.

There is a further structural consequence. With $\nu = 4$ and a 200 Hz band ceiling, the maximum expressible speed is 3000 rpm. **The deployed system structurally cannot report a Rotax 912 at cruise (4800–5500 rpm) or take-off (5800 rpm) power.** At 5000 rpm the firing frequency is 166.7 Hz (order 2, in band) and order 4 is 333 Hz (out of band), so the peak-picker would select the order-2 line and report 2500 rpm — half the truth. The system is, as configured, a low-speed and transient instrument.

13.2 Generalisation to other engines

`EVENTS_PER_CYCLE = 4` encodes an empirical property of one engine, one exhaust system, and one microphone placement. It does not transfer.

Engine	C	Cycle	Firing order	Likely dominant order
Rotax 912	4	4-stroke	2	2 or 4 (measured: both)
Lycoming O-320	4	4-stroke	2	2 or 4
Continental IO-550	6	4-stroke	3	3 or 6
Rotax 582	2	2-stroke	2	2 or 4
Turboprop (PT6A)	—	continuous	1 (shaft)	blade-pass

A six-cylinder engine fires at order 3, so a system tuned with $\nu = 4$ would report 3/4 of the true speed — a 25% error, small enough to look plausible and therefore more dangerous than the factor-of-two error. A two-stroke fires once per revolution *per cylinder*, so a twin fires at order 2 by a completely different route. Deploying this system on a different aircraft without recalibration would produce confidently wrong numbers.

The irony noted in Section 10 stands: `rpi_test_new.py` already contained the per-engine configuration hierarchy that would parameterise this correctly, and it was removed in the simplification to `rpi_test_final.py`.

13.3 Single dominant peak under multi-source interference

The `argmax` assumes the engine is the loudest thing in [10, 200] Hz. Realistic contaminants that are not:

- **Propeller blade-pass.** With the 2.43:1 Rotax reduction gearbox, a two-blade propeller at 5000 engine rpm turns at 2058 rpm, giving a shaft order at 34.3 Hz and blade-pass at 68.6 Hz — squarely inside the search band and, at some microphone positions, louder than the engine.
- **Airframe and boundary-layer noise.** Broadband but strongly low-frequency weighted, raising the floor exactly where the search band is.
- **Structural resonances.** A cowling or panel resonance is a fixed-frequency line that does *not* move with engine speed — and is therefore indistinguishable from an order in any single-snapshot analysis, though trivially distinguishable in a spectrogram.
- **Cabin sources.** Speech fundamentals run 85–255 Hz, overlapping the top of the band.

None of these are hypothetical for a cabin-mounted microphone in a light aircraft.

13.4 No temporal smoothing or state

The estimator is memoryless. Each chunk is analysed as though no chunk preceded it. This forfeits the strongest available prior: **engine speed is a continuous physical quantity with bounded rate of change.** A Rotax 912 cannot go from 1004 rpm to 501 rpm and back in ten seconds; the physical slew rate is limited by rotational inertia and available torque to a few hundred rpm per second at most.

Every one of the order-flip failures measured in Section 12 violates this constraint and would be rejected by even the crudest continuity test — a three-point median filter over the RPM sequence, at zero computational cost, would eliminate all three isolated flips in `1000_rpm.wav`. That such a cheap fix is not present is the most immediately actionable gap in the system.

13.5 Latency

The end-to-end latency budget:

Stage	Time
Recording	5.0 s
Queue wait	0–50 s (typically ~0)
WAV write + read	~0.05 s
Decimation and dual FFT	~0.1–0.5 s (Pi Zero)
UART transmit and ACK	0.01–8.0 s
Typical total	~5.2 s
Worst case	~63 s

An RPM reading therefore describes conditions that ended at least five seconds ago, and represents an average over that window rather than an instant. For trend logging this is irrelevant. For any application requiring prompt indication it is not, and the fix is overlapping windows: analysing a 5-second window every 1 second (80% overlap) would reduce reporting latency fivefold at five times the FFT cost, which after removing the redundant raw-signal FFT would still be cheaper than the current configuration.

13.6 Unintegrated components

Per the README, `EngineState` (TFLite), `TunableFilter`, and `RTAudioStreamSimulator` “are not yet integrated into `rpi_test_final.py`”. The consequences:

- The engine-state output is a 100 rpm threshold on a quantity that is itself unreliable, rather than a learned spectral classifier with 70-window voting.
- There is no adaptive narrowband tracking; every chunk searches the full band from scratch.
- Stateful components cannot be regression-tested against recorded audio despite the simulator existing to do exactly that.

Additionally, the library and deployed paths have diverged in ways that must be reconciled before integration: the library resamples to exactly 500 Hz while the script decimates to 501 Hz; the library’s `TunableFilter` caps at 150 Hz while the script searches to 200 Hz; and `TunableFilter`’s peak selection has the two defects documented in Section 8.

13.7 Implementation-level defects

Collected for completeness:

1. **Frequency/RPM mismatch in telemetry.** The transmitted `peak_freq` comes from the raw-signal analysis; the transmitted RPM from the decimated analysis (Section 8). Numerically close, structurally wrong, and it prevents any downstream cross-check.
2. **Redundant raw-signal FFT.** A 220,500-point transform per chunk, computed only to populate a telemetry field. This is the largest single cost in the pipeline.
3. **Rate truncation.** `44100 // 88 = 501` rather than `501.136`, a systematic -0.027% bias (Section 5). Invisible at Level 1 because the test audio is 48 kHz.
4. **Decimation factor 88 exceeds SciPy’s recommended maximum of 13** for single-stage IIR decimation (Section 5). Empirically fine via the SOS path, but fragile and more expensive than a cascade.
5. **Dead range guard.** `rpm > 10000` is unreachable; the reachable maximum is 3000 (Section 8).
6. **Hard-coded band edges.** 10 and 200 appear as literals inside the peak finder rather than as named constants, despite also governing the decimation design.
7. **Misleading default.** `RPM.__init__` defaults to `events_per_crankshaft_cycle=2` while every call site passes 4.
8. **Nyquist error in LowPassFilter/HighPassFilter.** `nyquist = sample_rate` instead of `sample_rate / 2` (Section 5). Not on the deployed path.
9. **TunableFilter peak selection.** Lowest-index rather than largest peak, and `distance=sample_rate` giving a ~ 50 Hz minimum separation (Section 8).
10. **RFFT divides by zero on silent input** during peak normalisation.
11. **Check-then-act race** on the global `ser` outside `uart_lock` (Section 9).
12. **No CRC over the header.** The start and length bytes are unprotected; a corrupted length is caught only by the end-byte check.

13.8 Validation gaps

1. **No tachometer ground truth** for any recording.
2. **Level 1 tests the wrong operating point** — whole files, not five-second chunks — and therefore passes on a configuration that fails in deployment.

3. **The volo1.wav tolerance $[0, 5000]$ rpm cannot fail.**
4. **No noise-robustness testing:** no additive noise sweep, no SNR characterisation, no interference cases.
5. **No transient testing:** no run-up or shutdown case, despite these being the conditions where a fixed-window estimator is weakest.

14 Future Research Directions

The directions below are ordered by the ratio of expected benefit to implementation cost, on the evidence of Section 12.

14.1 Harmonic-template order estimation

The single highest-value change. Replace the `argmax` with the comb estimator measured in Section 12:

$$\hat{\Omega} = \arg \max_{\Omega \in [\Omega_{\min}, \Omega_{\max}]} \sum_{o \in \mathcal{O}} \max_{|f - o\Omega/60| \leq 2\Delta f} |X(f)|,$$

with $\mathcal{O}$ derived from cylinder count and stroke cycle. On the repository’s own data this removed every order-flip failure on `1000_rpm.wav` and reduced the `vol01.wav` chunk-to-chunk standard deviation from 631 rpm to zero.

Refinements worth adding:

- **Log-magnitude or normalised scoring**, so that one dominant order cannot carry the score alone.
- **A sub-harmonic penalty** to suppress the octave error observed on chunk 1 of `1000_rpm.wav`: penalise candidates whose score is largely explained by a candidate at $\Omega/2$.
- **Coarse-to-fine search**: a 10 rpm grid followed by local refinement, cutting cost by an order of magnitude.
- **A confidence output**, naturally available as the ratio of the best score to the runner-up. This is what allows the downstream consumer to distinguish a 0.23 dB coin-flip from a decisive identification — the single most important piece of information the current system does not provide.

The `FeatureExtraction` interface makes this a drop-in replacement for `FrequencyPeakFinder`, and `EVENTS_PER_CYCLE` disappears from the configuration entirely, taking the generalisation problem of Section 13 with it.

14.2 Cepstral cross-check

The real cepstrum

$$c[q] = \mathcal{F}^{-1}\{\log |X(f)|\}$$

maps a uniformly spaced harmonic family to a single peak at quefrency $q = 1/f_{\text{spacing}}$. Since the problem is precisely to determine the family *spacing*, this is directly on point, cheap (one additional inverse FFT of an already-computed spectrum), and — importantly — *methodologically independent* of the comb estimator. Agreement between the two is strong evidence; disagreement is a useful flag for the confidence output.

14.3 Kalman and Vold–Kalman speed tracking

With the order ambiguity resolved, the next gain is temporal. Model the speed as a slowly varying state,

$$\begin{bmatrix} \Omega_k \\ \dot{\Omega}_k \end{bmatrix} = \begin{bmatrix} 1 & T \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \Omega_{k-1} \\ \dot{\Omega}_{k-1} \end{bmatrix} + \mathbf{w}_k, \quad z_k = \Omega_k + v_k,$$

with $T = 5$ s and process noise tuned to the engine’s physical slew limit. The constant-acceleration prior would reject every one of the Section 12 flips outright. Measurement noise R should be set from the estimator’s confidence output, so that ambiguous chunks are automatically down-weighted rather than accepted.

A three-point median filter over the RPM sequence is the trivial version of the same idea and should be implemented immediately, independent of anything else in this list: it costs nothing and removes all isolated flips.

The full Vold–Kalman filter [vold1995] becomes available once a reliable speed profile exists, since the profile can serve as the phase reference the method requires. With it, individual orders can be extracted as time-domain waveforms even during rapid run-ups — the natural substrate for the diagnostic work in the next subsection.

14.4 Tachless order tracking with ridge following

The full treatment from the literature [lu2019; peeters2019]: compute an STFT (the library’s STFT class already wraps it), extract order ridges as connected maxima trajectories, apply angular-acceleration constraints, integrate instantaneous frequency to phase, resample to the angle domain, and iterate. This handles transients that no fixed-window method can, and produces an angle-domain spectrum in which orders are sharp regardless of speed variation.

14.5 Integrating the TFLite classifier

Replace the 100 rpm threshold with `EngineState`, whose 70-window majority vote (Section 10) is a principled stabilisation of a noisy detector. Prerequisites: locating or retraining the model file, aligning the resample-versus-decimate divergence between library and script, and confirming `tflite_runtime` performance on the target Pi. The classifier’s multi-class output also opens the path from a binary on/off indicator to genuine fault detection — misfire, rough running, induction problems — which is the natural long-term direction for the project.

14.6 Sensor fusion with the vibration channel

`Sensors/` already contains a working ADXL345 interface with calibration. An accelerometer bolted to the engine mount observes the same order structure with a far better signal-to-interference ratio than a cabin microphone, because it is mechanically coupled to the source and is not exposed to propeller, wind, or cabin noise at all.

The most promising formulation is Shan and colleagues’ cross-correlation approach [shan2020]: components common to both the vibration and acoustic channels are reinforced while channel-specific interference is suppressed. Since a structural cowling resonance appears in only one channel and a genuine engine order in both, this directly attacks the interference limitation of Section 13.

14.7 Efficiency and robustness work

Small, certain improvements:

1. **Remove the redundant raw-signal FFT.** Report the decimated peak, and drop a 220,500-point transform per chunk. This alone is likely a 2–5x reduction in analyser CPU time.
2. **Use `rfft`.** Halves the remaining transform cost for real input.
3. **Multi-stage decimation.** Replace $M = 88$ with $2 \times 4 \times 11$: within SciPy’s recommendation, better conditioned, and cheaper because later stages operate at reduced rates.
4. **Carry the decimated rate as a float** to remove the 0.027% bias.
5. **Parabolic peak interpolation** for sub-bin resolution at three multiplies per estimate. This is worth more than the 3 rpm quantisation figure suggests: it also removes the scalloping-loss bias identified in Section 4, which can otherwise invert the measured ranking of two orders separated by less than 1.42 dB — and the margin in this system’s data is 0.23 dB.
6. **Overlapping windows** — 5 s analysed every 1 s — for 5x lower reporting latency, affordable once (1) and (2) are done.
7. **Physical range guard.** Replace the dead `rpm > 10000` check with the engine’s actual envelope, so out-of-envelope estimates are flagged rather than emitted.

14.8 Validation programme

The measurement gaps of Section 13 should be closed in this order:

1. **Tachometer ground truth.** Simultaneous audio and tachometer recording across idle, taxi, run-up, and cruise. Without this, accuracy cannot be *measured*, only argued.
2. **Chunk-level Level-1 testing.** Rewrite `test_pipeline.py` to process five-second chunks and assert on the *distribution* — median, spread, and order-flip rate — rather than a single whole-file value. This test would have caught the 21.4% failure rate before deployment.
3. **Tighten `vol01.wav`.** With the analysis of Section 12, the expected value is approximately 2527 rpm; the tolerance should be set accordingly, so that the test can fail.
4. **Noise robustness sweep.** Add pink and broadband noise at controlled SNRs and characterise degradation, giving a defensible operating envelope.
5. **Transient cases.** Synthesise or record run-ups and shutdowns, where fixed-window estimators are weakest.
6. **Extended ESP32 firmware.** The mock already serves as executable protocol documentation; a real implementation should reuse it directly.

15 References

The following works are cited in the text by bracketed key. Every entry was verified against the publisher of record.

[nyquist1928] Nyquist, H. (1928). “Certain Topics in Telegraph Transmission Theory.” *Transactions of the American Institute of Electrical Engineers*, **47**(2), 617–644. DOI: [10.1109/T-AIEE.1928.5055024](https://doi.org/10.1109/T-AIEE.1928.5055024)

[shannon1949] Shannon, C. E. (1949). “Communication in the Presence of Noise.” *Proceedings of the IRE*, **37**(1), 10–21. DOI: [10.1109/JRPROC.1949.232969](https://doi.org/10.1109/JRPROC.1949.232969)

[oppenheim2010] Oppenheim, A. V. and Schafer, R. W. (2010). *Discrete-Time Signal Processing*, 3rd edition. Prentice Hall / Pearson, Upper Saddle River, NJ. ISBN 978-0-13-198842-2.

[cooley1965] Cooley, J. W. and Tukey, J. W. (1965). “An Algorithm for the Machine Calculation of Complex Fourier Series.” *Mathematics of Computation*, **19**(90), 297–301. DOI: [10.1090/S0025-5718-1965-0178586-1](https://doi.org/10.1090/S0025-5718-1965-0178586-1)

[harris1978] Harris, F. J. (1978). “On the Use of Windows for Harmonic Analysis with the Discrete Fourier Transform.” *Proceedings of the IEEE*, **66**(1), 51–83. DOI: [10.1109/PROC.1978.10837](https://doi.org/10.1109/PROC.1978.10837)

[butterworth1930] Butterworth, S. (Admiralty Research Laboratory) (1930). “On the Theory of Filter Amplifiers.” *Experimental Wireless and the Wireless Engineer*, **7**, 536–541 (October 1930). Quotations in Section 5 were taken from the original published scan.

[crochiere1981] Crochiere, R. E. and Rabiner, L. R. (1981). “Interpolation and Decimation of Digital Signals — A Tutorial Review.” *Proceedings of the IEEE*, **69**(3), 300–331. DOI: [10.1109/PROC.1981.11969](https://doi.org/10.1109/PROC.1981.11969)

[crochiere1983] Crochiere, R. E. and Rabiner, L. R. (1983). *Multirate Digital Signal Processing*. Prentice-Hall, Englewood Cliffs, NJ.

[vold1995] Vold, H. and Leuridan, J. (1995). “High Resolution Order Tracking at Extreme Slew Rates Using Kalman Tracking Filters.” *Shock and Vibration*, **2**(6), 507–515. DOI: [10.3233/SAV-1995-2609](https://doi.org/10.3233/SAV-1995-2609) (originally SAE Technical Paper 931288, 1993).

[randall2011] Randall, R. B. (2011). *Vibration-based Condition Monitoring: Industrial, Aerospace and Automotive Applications*. John Wiley and Sons, Chichester. ISBN 978-0-470-74785-8. DOI: [10.1002/9780470977668](https://doi.org/10.1002/9780470977668)

[randallantoni2011] Randall, R. B. and Antoni, J. (2011). “Rolling Element Bearing Diagnostics — A Tutorial.” *Mechanical Systems and Signal Processing*, **25**(2), 485–520. DOI: [10.1016/j.ymssp.2010.07.017](https://doi.org/10.1016/j.ymssp.2010.07.017)

[antoni2009] Antoni, J. (2009). “Cyclostationarity by Examples.” *Mechanical Systems and Signal Processing*, **23**(4), 987–1036. DOI: [10.1016/j.ymssp.2008.10.010](https://doi.org/10.1016/j.ymssp.2008.10.010)

[lu2019] Lu, S., Yan, R., Liu, Y. and Wang, Q. (2019). “Tachless Speed Estimation in Order Tracking: A Review With Application to Rotating Machine Fault Diagnosis.” *IEEE Transactions on Instrumentation and Measurement*, **68**(7), 2315–2332. DOI: [10.1109/TIM.2019.2902806](https://doi.org/10.1109/TIM.2019.2902806)

[peeters2019] Peeters, C., Leclere, Q., Antoni, J., Lindahl, P., Donnal, J., Leeb, S. and Helsen, J. (2019). “Review and Comparison of Tachless Instantaneous Speed Estimation Methods on

Experimental Vibration Data.” *Mechanical Systems and Signal Processing*, **129**, 407–436. DOI: [10.1016/j.ymssp.2019.02.031](https://doi.org/10.1016/j.ymssp.2019.02.031)

[shan2020] Shan, X., Tang, L., Wen, H., Martinek, R. and Smulko, J. (2020). “Analysis of Vibration and Acoustic Signals for Noncontact Measurement of Engine Rotation Speed.” *Sensors*, **20**(3), 683. DOI: [10.3390/s20030683](https://doi.org/10.3390/s20030683)

[rife1974] Rife, D. C. and Boorstyn, R. R. (1974). “Single-Tone Parameter Estimation from Discrete-Time Observations.” *IEEE Transactions on Information Theory*, **20**(5), 591–598. DOI: [10.1109/TIT.1974.1055282](https://doi.org/10.1109/TIT.1974.1055282)

[peterson1961] Peterson, W. W. and Brown, D. T. (1961). “Cyclic Codes for Error Detection.” *Proceedings of the IRE*, **49**(1), 228–235. DOI: [10.1109/JRPROC.1961.287814](https://doi.org/10.1109/JRPROC.1961.287814)

[koopman2004] Koopman, P. and Chakravarty, T. (2004). “Cyclic Redundancy Code (CRC) Polynomial Selection for Embedded Networks.” In *Proceedings of the International Conference on Dependable Systems and Networks (DSN 2004)*, Florence, Italy, 145–154. DOI: [10.1109/DSN.2004.1311885](https://doi.org/10.1109/DSN.2004.1311885)

[dijkstra1968] Dijkstra, E. W. (1968). “Cooperating Sequential Processes.” In F. Genuys (ed.), *Programming Languages: NATO Advanced Study Institute*, Academic Press, London, 43–112. (Originally circulated as EWD123, 1965; reprinted in P. Brinch Hansen (ed.), *The Origin of Concurrent Programming*, Springer, 2002, 65–138.)

[numpy] Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). “Array Programming with NumPy.” *Nature*, **585**, 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2)

[scipy] Virtanen, P., Gommers, R., Oliphant, T. E., et al. (2020). “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python.” *Nature Methods*, **17**(3), 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2)

[scipydecimate] SciPy Developers (2024). “`scipy.signal.decimate`.” *SciPy Reference Guide*, version 1.13. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.decimate.html>

[rotax912] BRP-Rotax GmbH and Co KG. “912 ULS / S — Technical Data.” Rotax Aircraft Engines. <https://www.flyrotax.com/products/912-uls-s> (4 cylinders, four-stroke, 1352 cm³, bore 84.0 mm, stroke 61.0 mm, 73.5 kW / 100 hp at 5800 rpm, gearbox reduction ratio 2.43).

[tecnam] Costruzioni Aeronautiche Tecnam. “P92 Echo — Specification and Description.” <https://www.tecnam.com/aircraft/p92-echo-light/>

16 Appendix A: Reproducing the Measurements

Every quantitative result in Sections 11 and 12 was produced by executing the repository’s own pipeline classes. The procedure is recorded here so the numbers can be independently checked.

Environment. Python 3, NumPy 2.0.0, SciPy 1.13.1.

Level-1 reproduction. Import `Datum`, `FeatureEngineeringPipeline`, `Decimation`, `FrequencyPeakFinder`, `RPM`, `DerivedDataKey`, `DECIMATED_RATE`, and `EVENTS_PER_CYCLE` from `rpi_test_final`; assemble the pipeline exactly as `analyzer()` does; read each WAV with `scipy.io.wavfile.read`, take channel 0 if stereo, cast to `float32`, and run. Read back `RPM`, `ENGINE_STATUS`, `FREQUENCY_PEAK`, and `DECIMATED_FREQUENCY_PEAK`.

Order-amplitude tables (Sections 7 and 12). Decimate the whole file with `scipy.signal.decimate(d, sr//500, ftype='iir')`, apply `np.hanning`, take `np.abs(np.fft.rfft(...))`, and normalise by the maximum over $[10, 200]$ Hz. For each order o in the half-order family, report the maximum magnitude within ± 3 bins of $o \cdot f_{0.5}$. The half-order bases were determined as one-quarter of the dominant peak: $66.944/8 = 8.368$ Hz for `1000_rpm.wav` and $84.236/4 = 21.059$ Hz for `vol01.wav`.

Chunk-level analysis (Section 12). Partition each file into non-overlapping $5f_s$ -sample blocks and run the identical decimate–window–FFT–argmax chain on each independently, exactly as the deployed recorder and analyser do.

Harmonic-template estimator (Section 12). For each candidate Ω on a 1 rpm grid over $[400, 3500]$, sum the maximum spectral magnitude within ± 2 bins of $o\Omega/60$ for each $o \in \{0.5, 1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5, 6\}$, skipping orders falling outside $[5, 220]$ Hz. The spectrum is normalised to unit maximum over $[5, 220]$ Hz before scoring.

Filter response measurement (Section 5). Design `scipy.signal.cheby1(8, 0.05, 0.8/M, output='sos')` and evaluate with `scipy.signal.sosfreqz` at 400,000 points. Effective attenuation under `zero_phase=True` is twice the single-pass value in decibels. The second-order sections form must be used: evaluating the same design in `(b, a)` transfer-function form produces numerical artefacts of several decibels at these extreme normalised cutoffs.